From numbers to AI

ARTIFICIAL INTELLIGENCE

인공지능이란 무엇인가

CONTENT

01 AI란 무엇인가

02 SYMBOLIC AI VS STATISTICAL AI

03 기계학습이란 무엇인가

04 분류와 회귀

05 군집화와 차원 축소

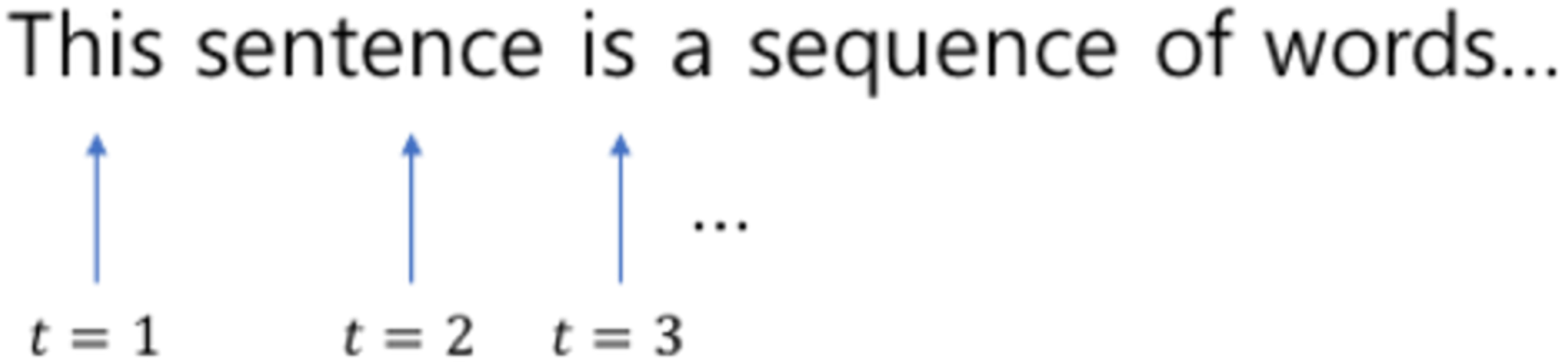
06 딥러닝

07 **강화학습**

08 인공지능의 활용과 윤리

RECURRENT NEURAL NETWORK

이미지는 정적인 데이터... 소리나 영상은? 문장은?

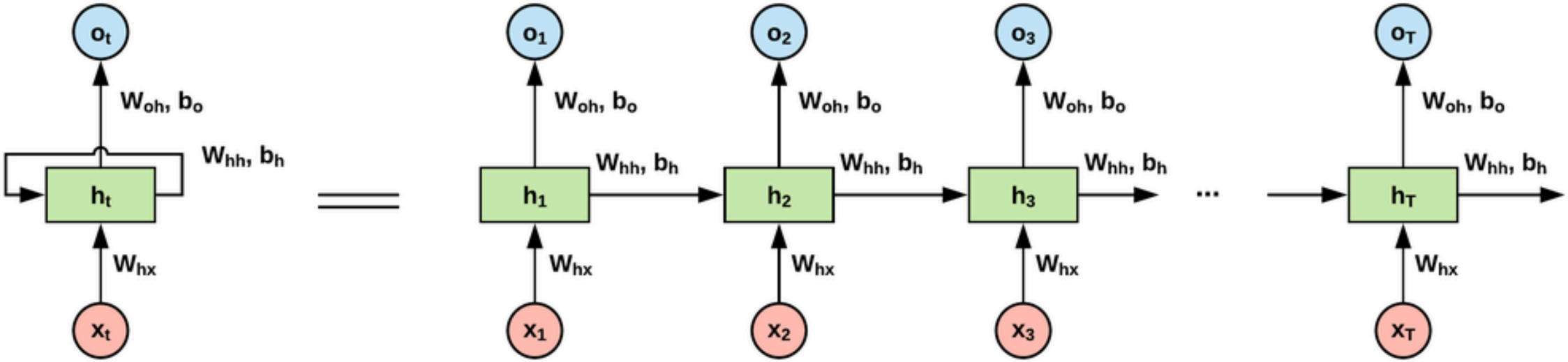


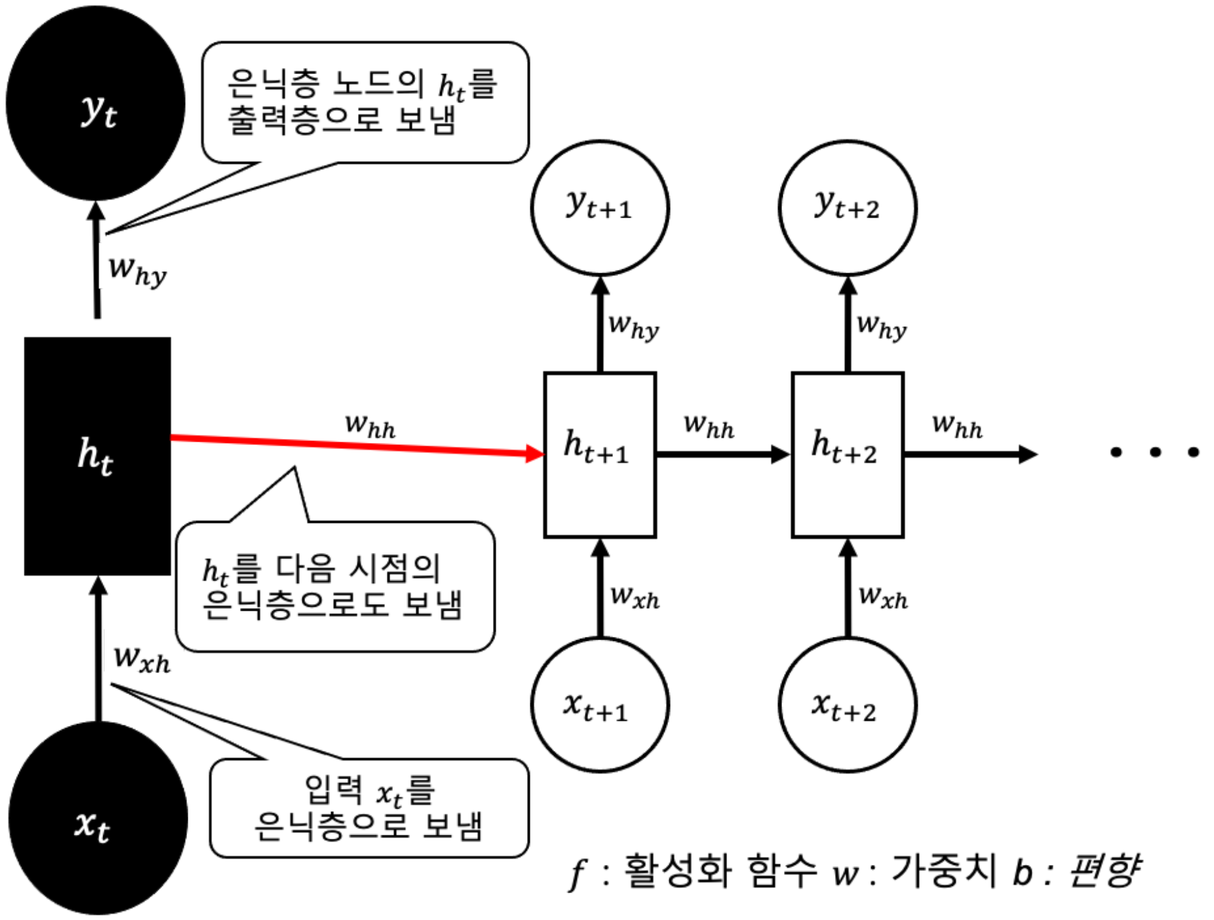
주가 데이터

음성 신호

RECURRENT?

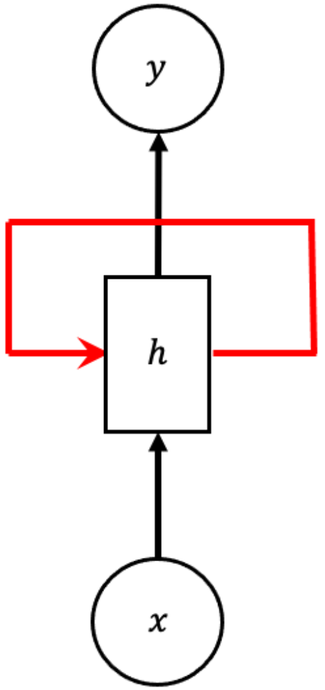
반복되는..



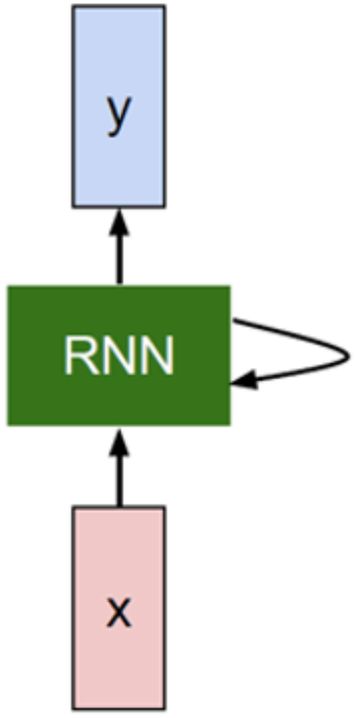


f : 활성화 함수 w : 가중치 b : *편향*

$$\mathrm{h_t} = f\,(w_{hh} \times h_{t-1} + w_{xh} \times x_t + b)$$
$$y_t = softmax(w_{hy} \times \mathrm{h_t})$$

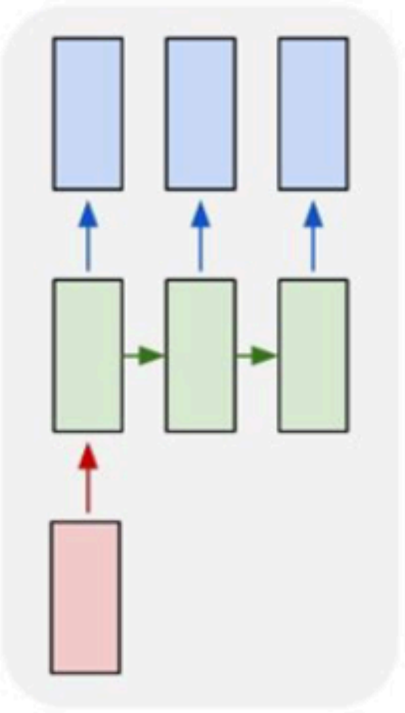


은닉층의 결과가 은닉층으로 들어가도록 연결되어 있어,
이전의 값들을 기억하며 전달하는 형태



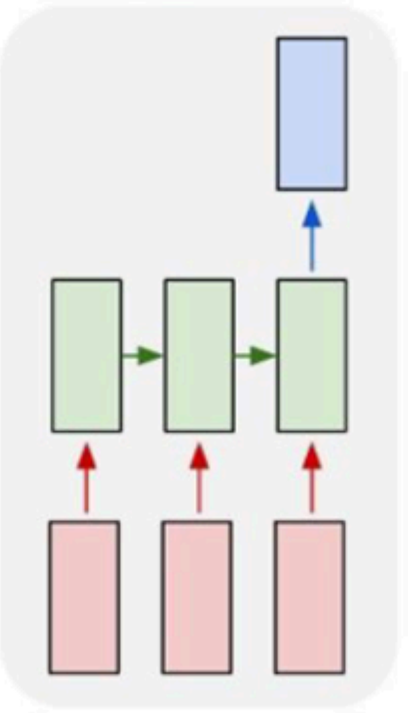
<기본구조>

one to many



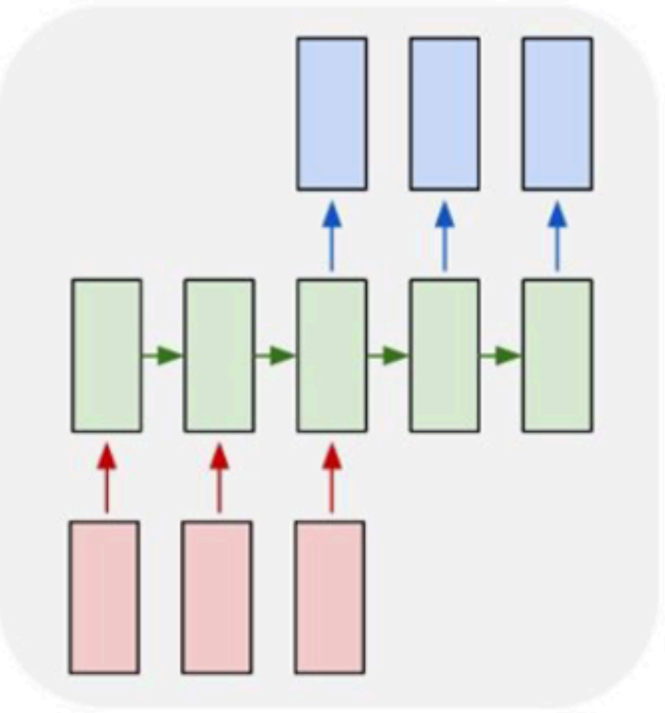
<사진설명 붙이기>
사진 → 단어들

many to one



<감성분석>
단어들 → 감성점수

many to many



<번역>
단어들 → 단어들

```
class RNN(nn.Module):

    # you can also accept arguments in your model constructor
    def __init__(self, data_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size
        input_size = data_size + hidden_size

        self.i2h = nn.Linear(input_size, hidden_size)
        self.h2o = nn.Linear(hidden_size, output_size)

    def forward(self, data, last_hidden):
        input = torch.cat((data, last_hidden), 1)
        hidden = self.i2h(input)
        output = self.h2o(hidden)
        return hidden, output


rnn = RNN(50, 20, 10)
```

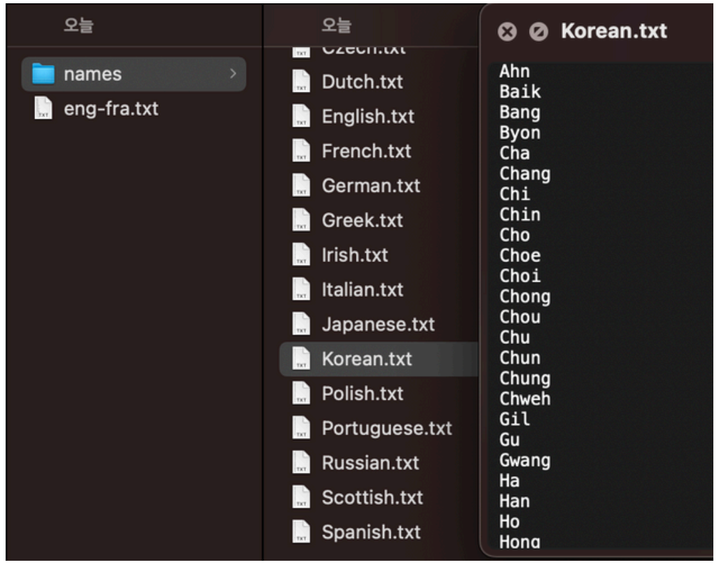
```
loss_fn = nn.MSELoss()

batch_size = 10
TIMESTEPS = 5

# Create some fake data
batch = torch.randn(batch_size, 50)
hidden = torch.zeros(batch_size, 20)
target = torch.zeros(batch_size, 10)

loss = 0
for t in range(TIMESTEPS):
    # yes! you can reuse the same network several times,
    # sum up the losses, and call backward!
    hidden, output = rnn(batch, hidden)
    loss += loss_fn(output, target)
loss.backward()
```

HTTPS://TUTORIALS.PYTORCH.KR/INTERMEDIATE/CHAR_RNN_CLASSIFICATION_TUTORIAL.HTML

데이터 준비

• 참고

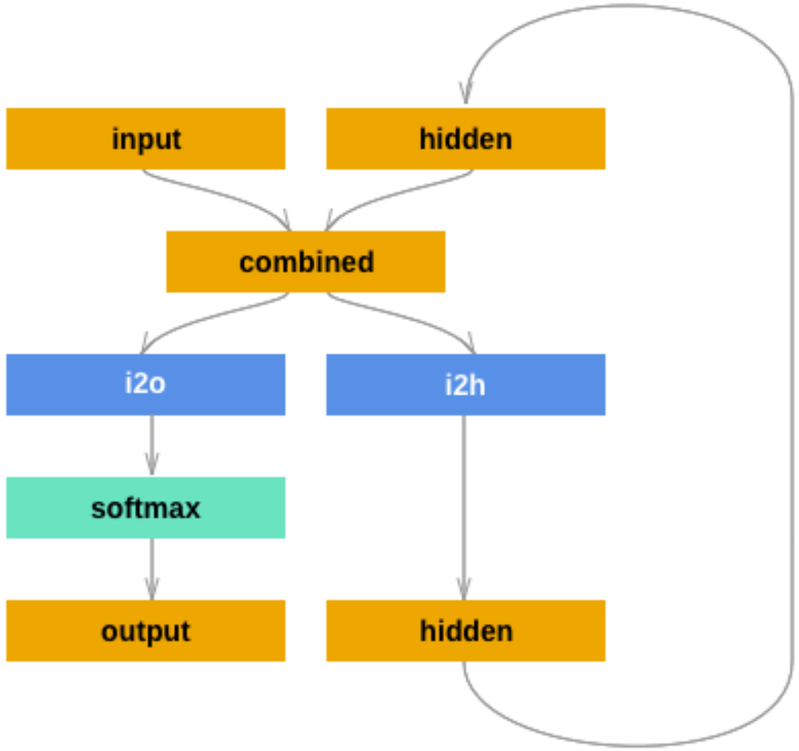
여기 에서 데이터를 다운로드 받고 현재 디렉토리에 압축을 푸십시오.




```
print(category_lines['Italian'][:5])
```

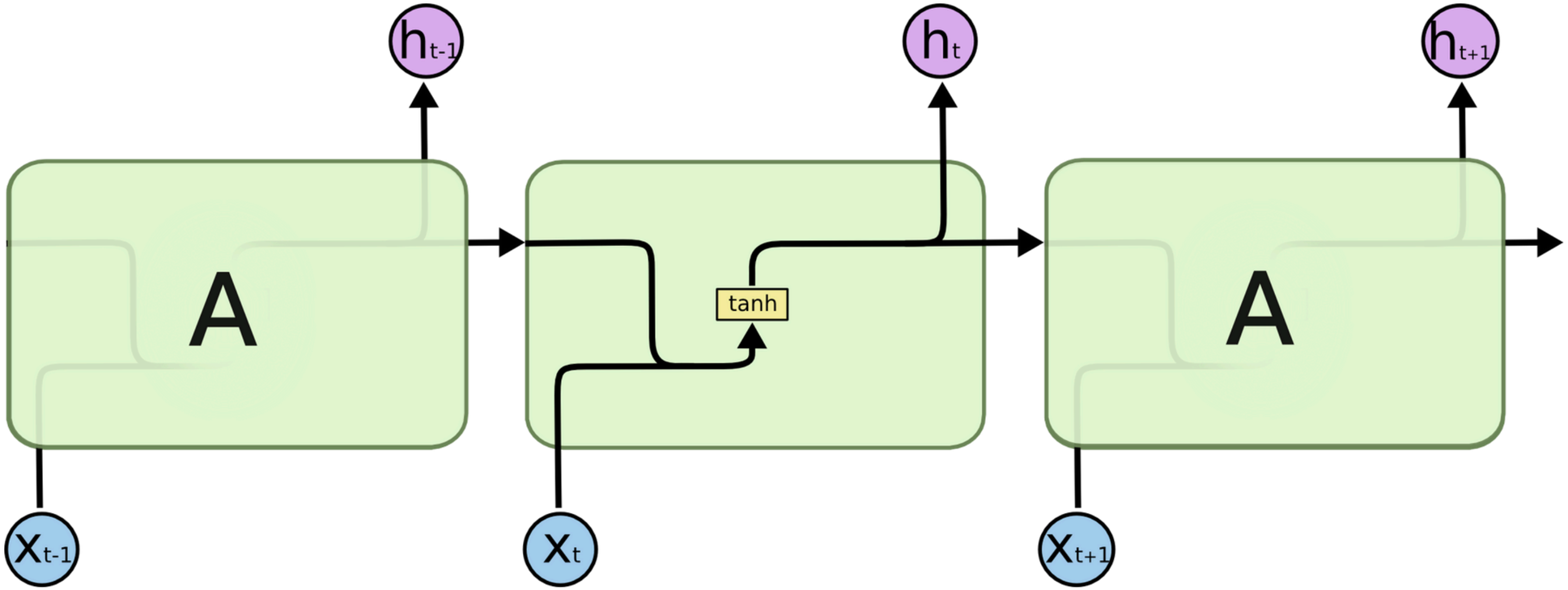
Out:

```
['Abandonato', 'Abatangelo', 'Abatantuono', 'Abate', 'Abategiovanni']
```



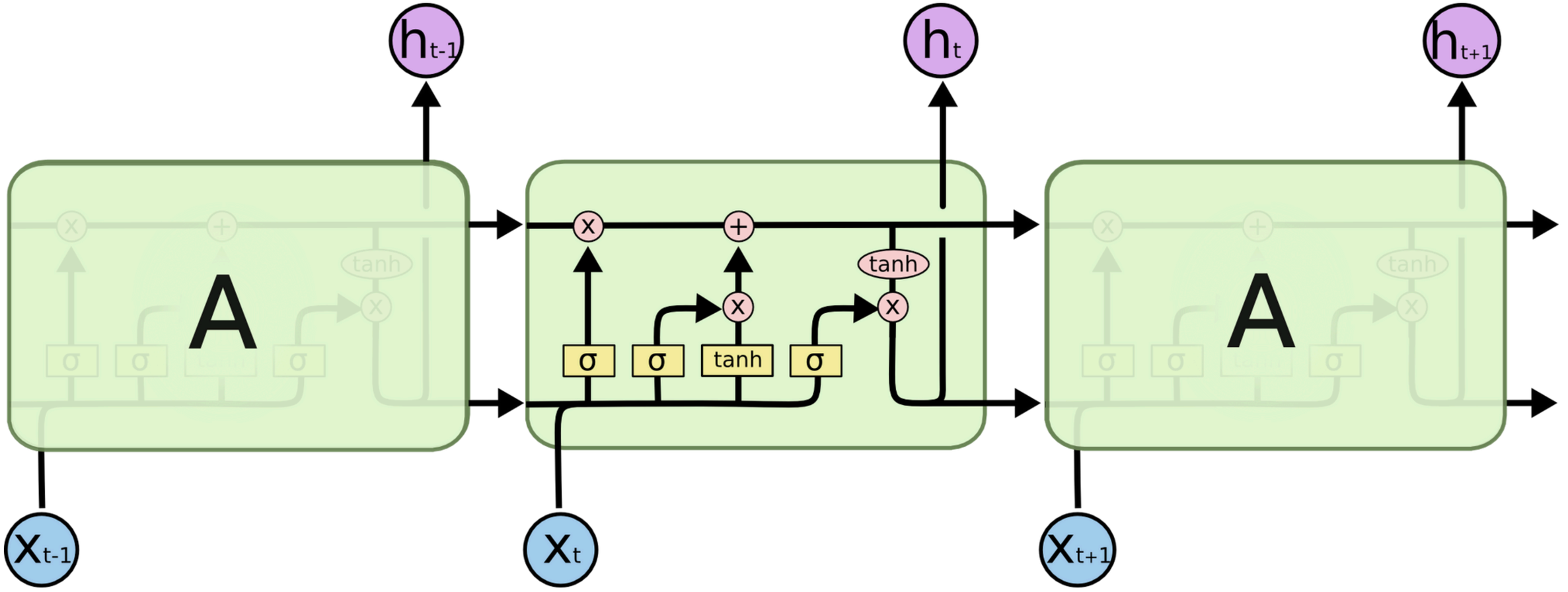

```
import torch.nn as nn

class RNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(RNN, self).__init__()

        self.hidden_size = hidden_size

        self.i2h = nn.Linear(input_size + hidden_size, hidden_size)
        self.i2o = nn.Linear(input_size + hidden_size, output_size)
        self.softmax = nn.LogSoftmax(dim=1)

    def forward(self, input, hidden):
        combined = torch.cat((input, hidden), 1)
        hidden = self.i2h(combined)
        output = self.i2o(combined)
        output = self.softmax(output)
        return output, hidden

    def initHidden(self):
        return torch.zeros(1, self.hidden_size)

n_hidden = 128
rnn = RNN(n_letters, n_hidden, n_categories)
```

```
$ python predict.py Schmidhuber
(-0.19) German
(-2.48) Czech
(-2.68) Dutch
```

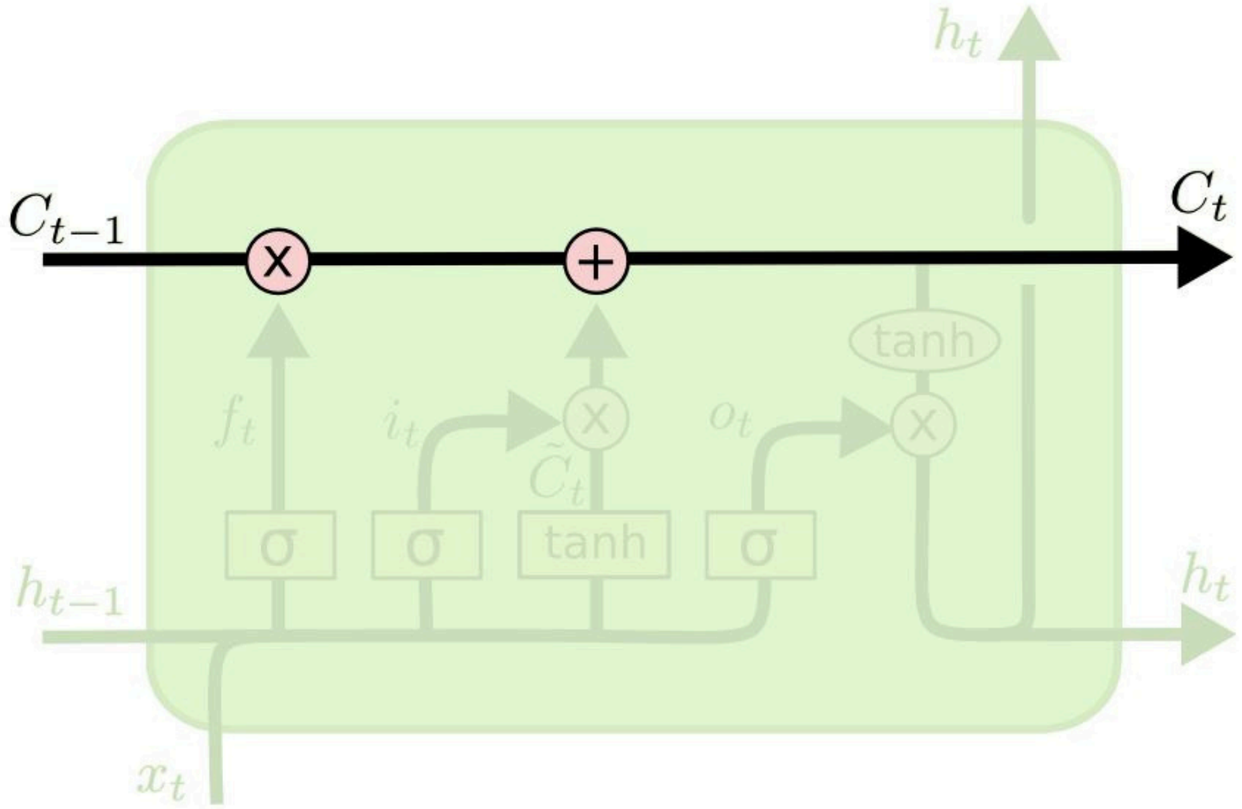

h_{t-1}
h_t
h_{t+1}
A
tanh
A
X_{t-1}
X_t
X_{t+1}

5-2. LONG SHORT TERM MEMORY

고도화!

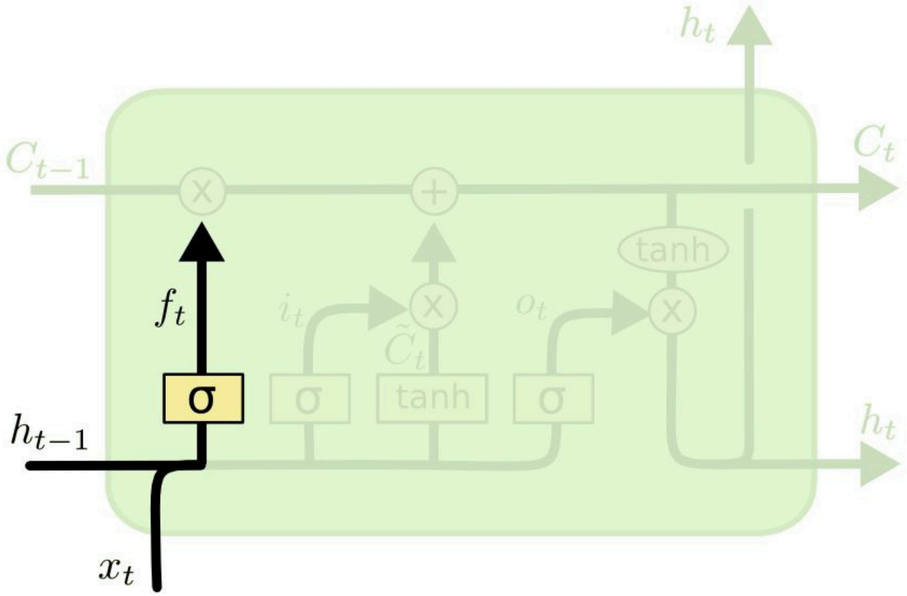


CELL STATE



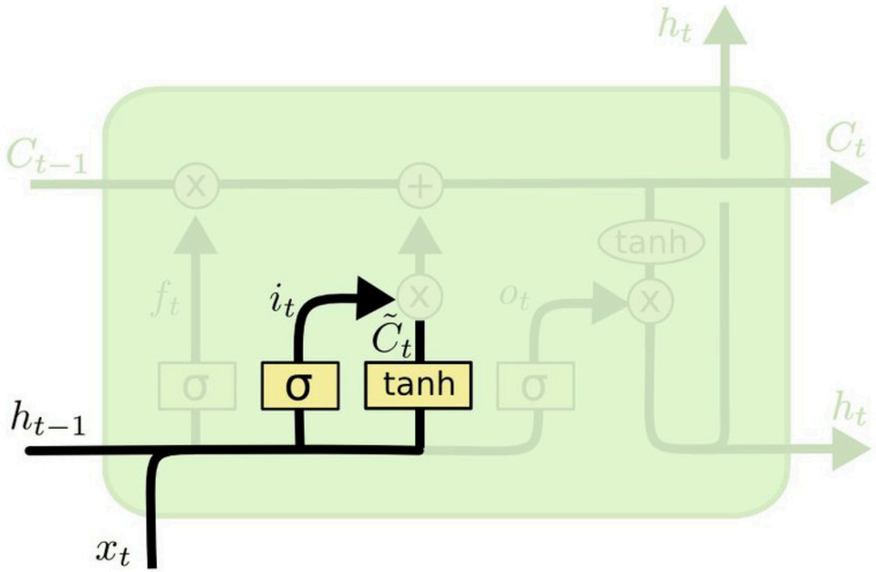
HTTPS://WIKIDOCS.NET/152773

FORGET GATE



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

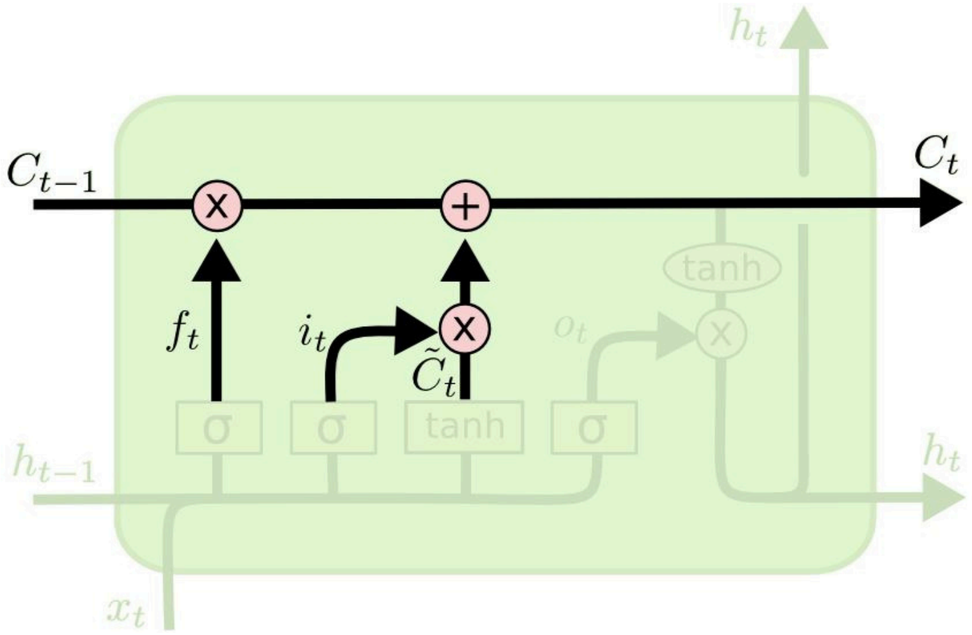
INPUT GATE



$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

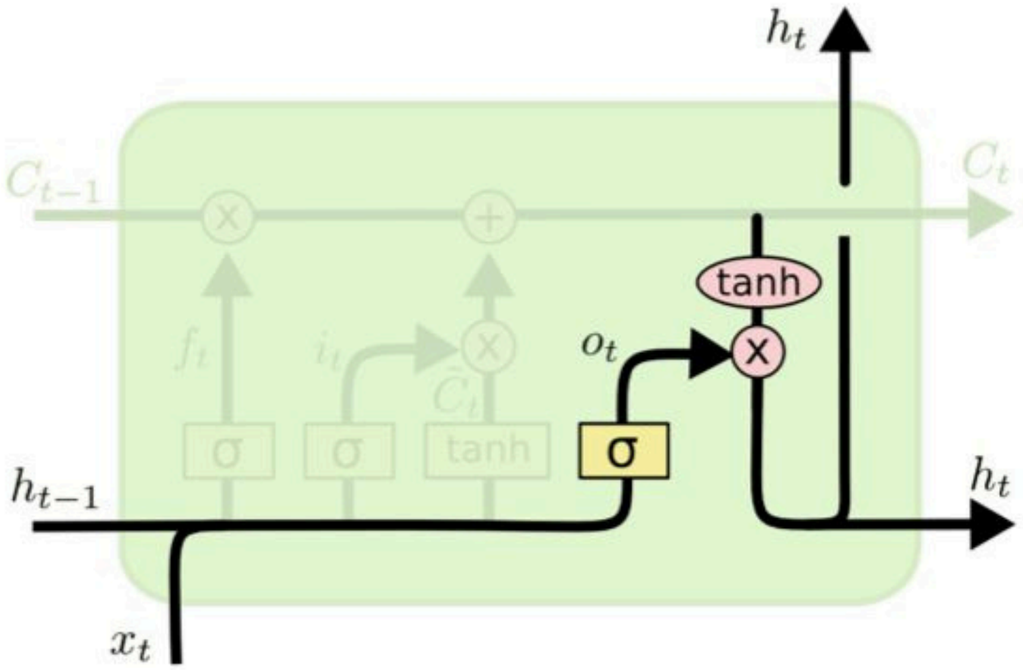
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

UPDATE



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

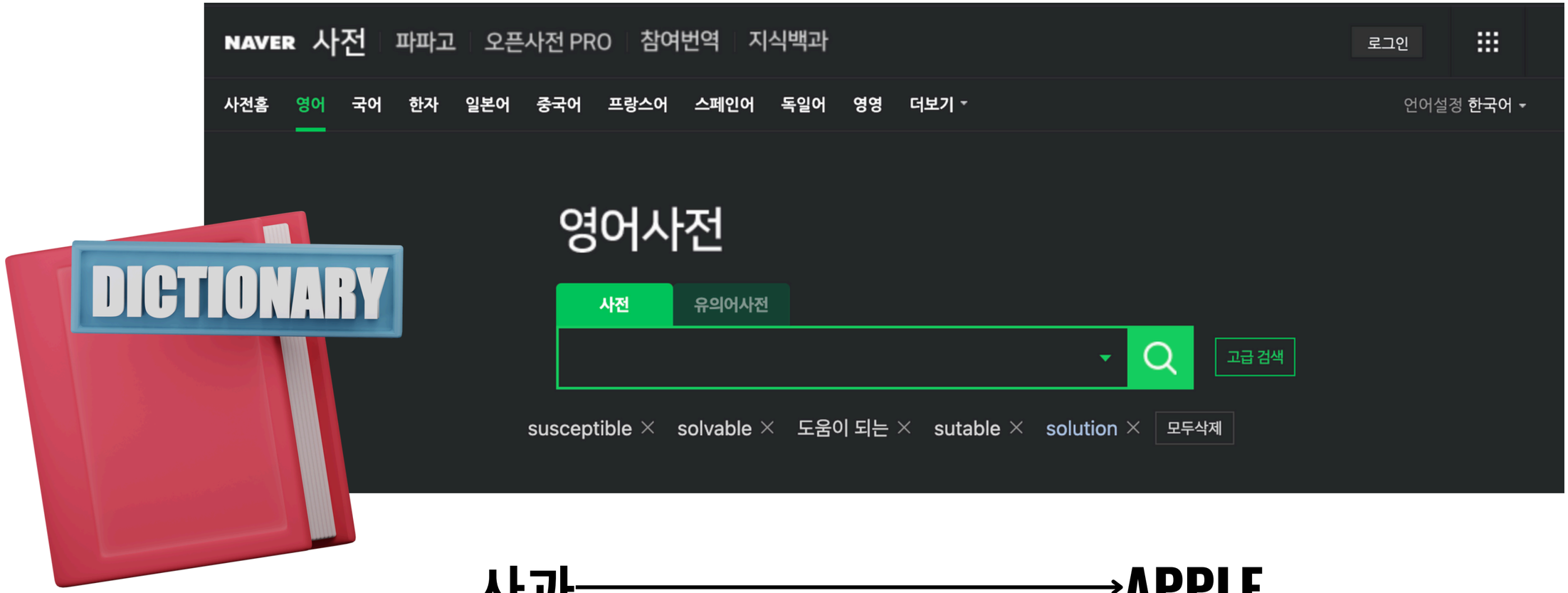
OUTPUT



$$o_t = \sigma\left(W_o\ [h_{t-1}, x_t]\ +\ b_o\right)$$

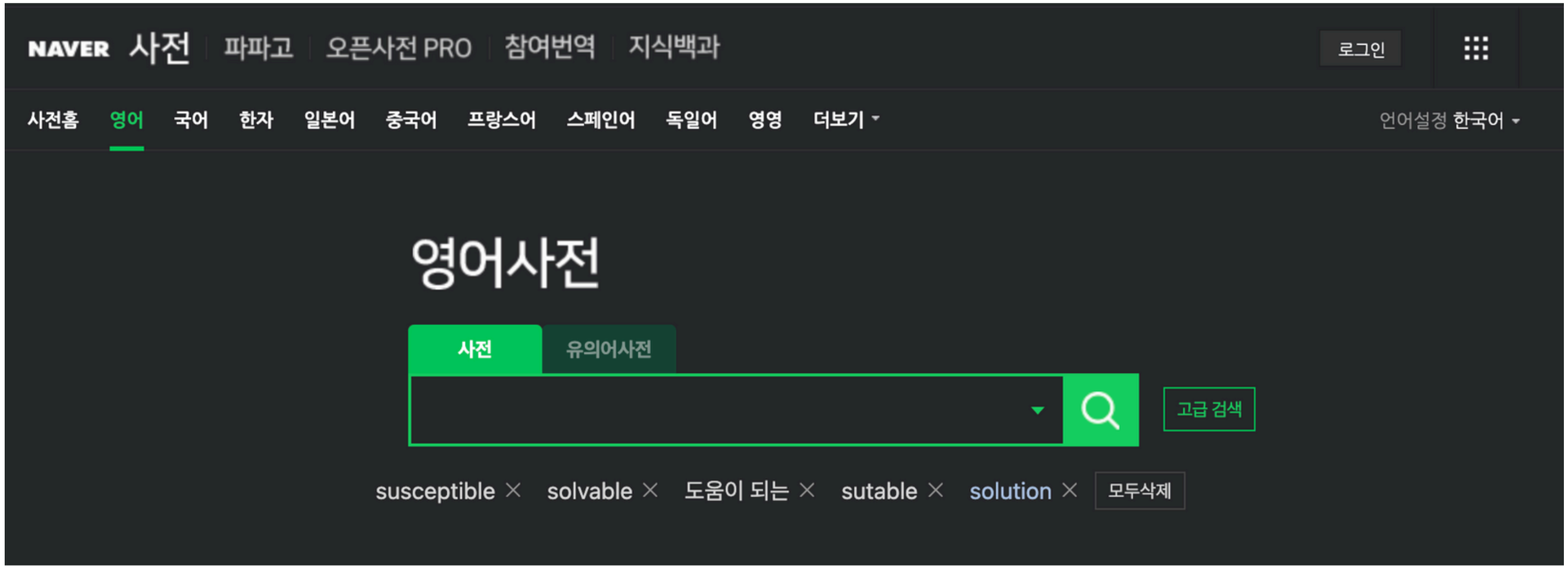
$$h_t = o_t * \tanh\left(C_t\right)$$

5-3. TRANSFORMER



사과 ⟶ APPLE

KEY DICT{KEY:VALUE} VALUE



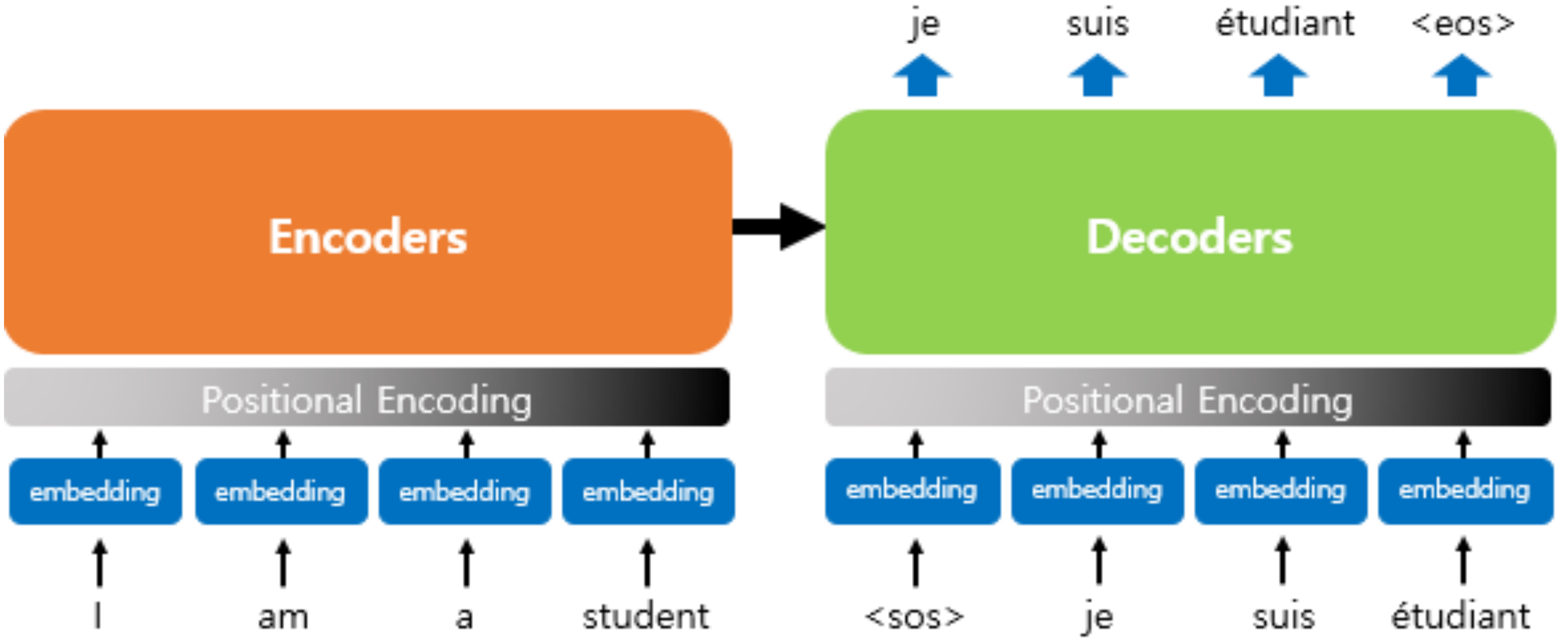
나는 /사과를 /좋아해 ⟶ I /LOVE / APPLE

QUERY OUTPUT

“나는 너를 좋아___”

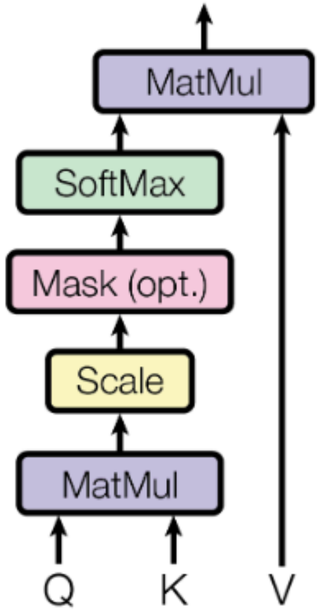
의미와 위치 “나는 ____ 좋아해”

“세상에서 쫄면이 제일 맛있어 김밥은 그 ___이야”

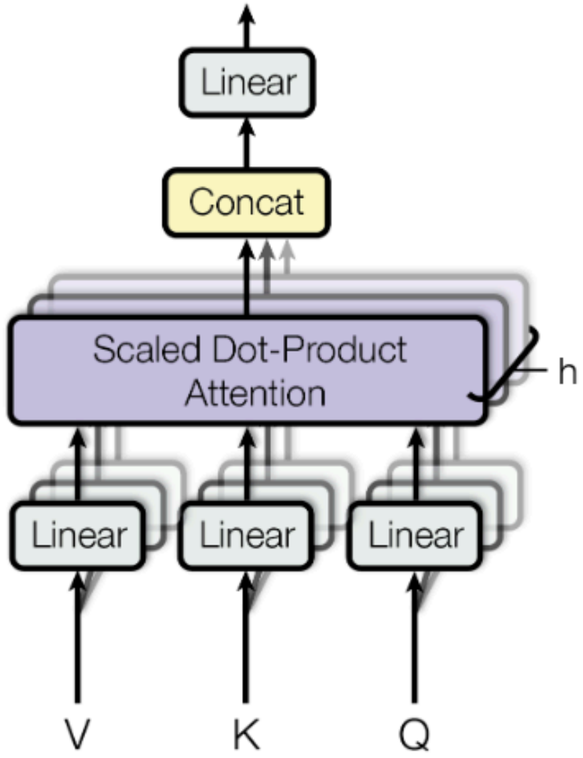

je
suis
étudiant
<eos>
Encoders
Decoders
Positional Encoding
Positional Encoding
embedding
embedding
embedding
embedding
embedding
embedding
embedding
embedding
I
am
a
student
<sos>
je
suis
étudiant

연결관계: Query, key, value

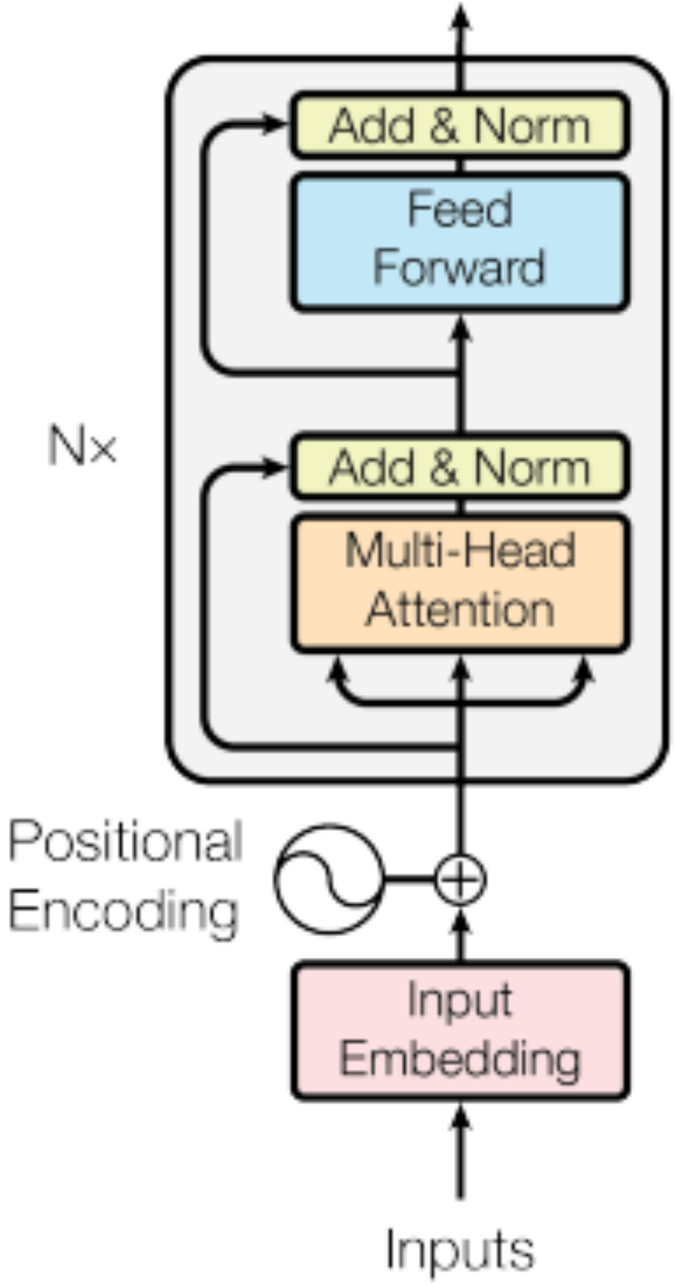
Scaled Dot-Product Attention



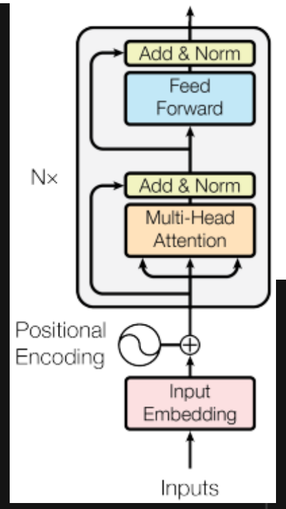
Multi-Head Attention



ENCODER

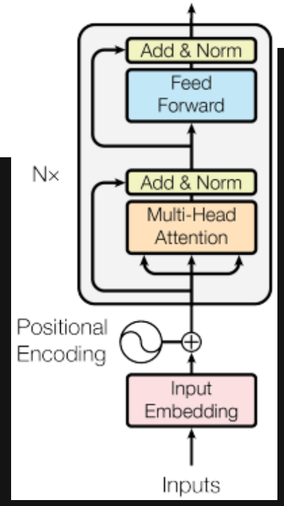

Add & Norm
Feed Forward
Nx
Add & Norm
Multi-Head Attention
Positional Encoding
Input Embedding
Inputs

```python
class Encoder(nn.Module):
    def __init__(self,
                 input_dim,
                 hid_dim,
                 n_layers,
                 n_heads,
                 pf_dim,
                 dropout,
                 device,
                 max_length = 100):
        super().__init__()

        self.device = device

        ''' Input Embedding '''
        self.tok_embedding = nn.Embedding(input_dim, hid_dim)
        # <sos> token부터 문장의 최대 길이로 설정한 100까지 positional embedding을 함.
        self.pos_embedding = nn.Embedding(max_length, hid_dim)

        ''' Multiple Encoder '''
        # 논문에서는 N=6개의 encoder layer를 사용하였음
        self.layers = nn.ModuleList([EncoderLayer(hid_dim,
                                                  n_heads,
                                                  pf_dim,
                                                  dropout,
                                                  device)
                                     for _ in range(n_layers)])

        self.dropout = nn.Dropout(dropout)

        self.scale = torch.sqrt(torch.FloatTensor([hid_dim])).to(device)
```



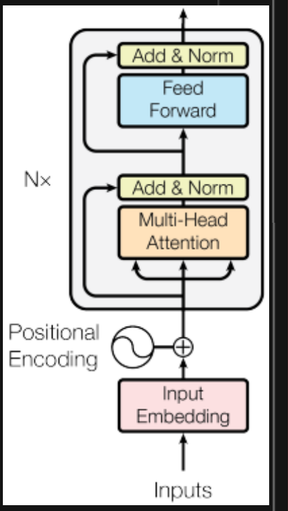

```python
def forward(self, src, src_mask):

    #src = [batch size, src len]
    #src_mask = [batch size, 1, 1, src len]

    batch_size = src.shape[0]
    src_len = src.shape[1]

    pos = torch.arange(0, src_len).unsqueeze(0).repeat(batch_size, 1).to(self.device)

    #pos = [batch size, src len]

    # src 문장의 embedding vector와 positional vector를 더해줌
    src = self.dropout((self.tok_embedding(src) * self.scale) + self.pos_embedding(pos))

    #src = [batch size, src len, hid dim]

    for layer in self.layers:
        src = layer(src, src_mask)

    #src = [batch size, src len, hid dim]

    return src
```

```python
class EncoderLayer(nn.Module):
    def __init__(self,
                 hid_dim,
                 n_heads,
                 pf_dim,
                 dropout,
                 device):
        super().__init__()

        ''' Multi Head self-Attention '''
        self.self_attention = MultiHeadAttentionLayer(hid_dim, n_heads, dropout, device)
        self.self_attn_layer_norm = nn.LayerNorm(hid_dim)

        ''' Positional FeedForward Layer'''
        self.positionwise_feedforward = PositionwiseFeedforwardLayer(hid_dim,
                                                                      pf_dim,
                                                                      dropout)
        self.ff_layer_norm = nn.LayerNorm(hid_dim)

        self.dropout = nn.Dropout(dropout)
```




```python
    def forward(self, src, src_mask):

        #src = [batch size, src len, hid dim]
        #src_mask = [batch size, 1, 1, src len]

        #self attention
        _src, _ = self.self_attention(src, src, src, src_mask)

        #dropout, residual connection and layer norm
        src = self.self_attn_layer_norm(src + self.dropout(_src))

        #src = [batch size, src len, hid dim]

        #positionwise feedforward
        _src = self.positionwise_feedforward(src)

        #dropout, residual and layer norm
        src = self.ff_layer_norm(src + self.dropout(_src))

        #src = [batch size, src len, hid dim]

        return src
```

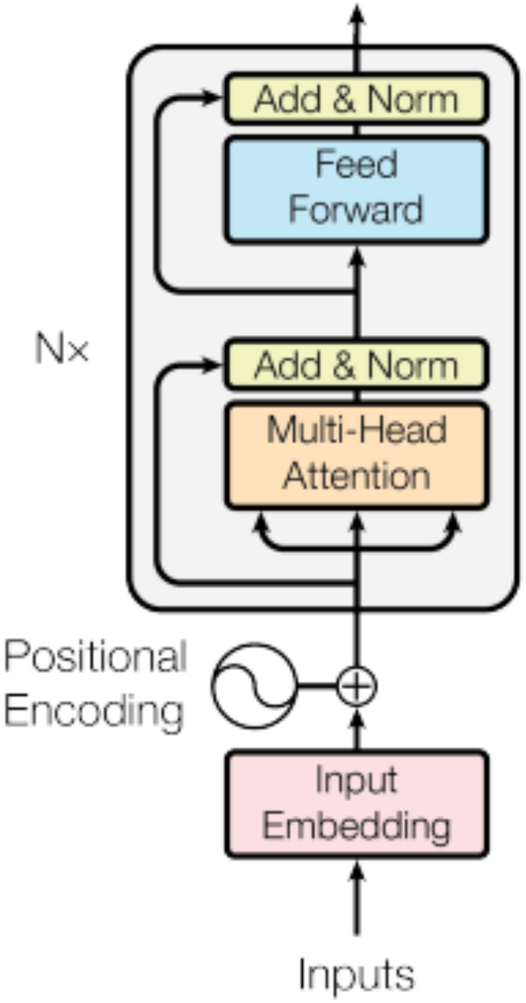
```python
class MultiHeadAttentionLayer(nn.Module):
    def __init__(self, hid_dim, n_heads, dropout, device):
        super().__init__()

        assert hid_dim % n_heads == 0

        self.hid_dim = hid_dim
        self.n_heads = n_heads
        self.head_dim = hid_dim // n_heads

        # encoder의 Q, K, V는 모두 같은 space에서 옵니다.

        self.fc_q = nn.Linear(hid_dim, hid_dim)
        self.fc_k = nn.Linear(hid_dim, hid_dim)
        self.fc_v = nn.Linear(hid_dim, hid_dim)

        self.fc_o = nn.Linear(hid_dim, hid_dim)

        self.dropout = nn.Dropout(dropout)

        self.scale = torch.sqrt(torch.FloatTensor([self.head_dim])).to(device)
```



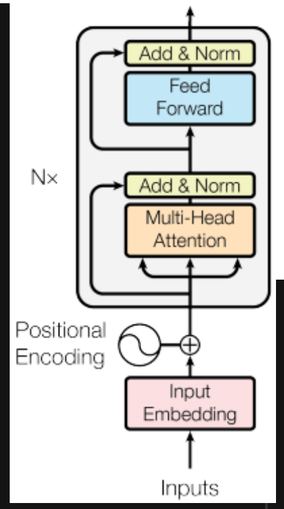
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

```python
    batch_size = query.shape[0]
    #query = [batch size, query len, hid dim]
    #key = [batch size, key len, hid dim]
    #value = [batch size, value len, hid dim]

    Q = self.fc_q(query)
    K = self.fc_k(key)
    V = self.fc_v(value)
    #Q = [batch size, query len, hid dim]
    #K = [batch size, key len, hid dim]
    #V = [batch size, value len, hid dim]

    Q = Q.view(batch_size, -1, self.n_heads, self.head_dim).permute(0, 2, 1, 3)
    K = K.view(batch_size, -1, self.n_heads, self.head_dim).permute(0, 2, 1, 3)
    V = V.view(batch_size, -1, self.n_heads, self.head_dim).permute(0, 2, 1, 3)
    #Q = [batch size, n heads, query len, head dim]
    #K = [batch size, n heads, key len, head dim]
    #V = [batch size, n heads, value len, head dim]

    energy = torch.matmul(Q, K.permute(0, 1, 3, 2)) / self.scale
    #energy = [batch size, n heads, query len, key len]

    if mask is not None:
        energy = energy.masked_fill(mask == 0, -1e10)

    attention = torch.softmax(energy, dim = -1)
    #attention = [batch size, n heads, query len, key len]

    x = torch.matmul(self.dropout(attention), V)
    #x = [batch size, n heads, query len, head dim]

    x = x.permute(0, 2, 1, 3).contiguous()
    #x = [batch size, query len, n heads, head dim]

    x = x.view(batch_size, -1, self.hid_dim)
    #x = [batch size, query len, hid dim]

    x = self.fc_o(x)
    #x = [batch size, query len, hid dim]

    return x, attention
```

```python
class PositionwiseFeedforwardLayer(nn.Module):
    def __init__(self, hid_dim, pf_dim, dropout):
        super().__init__()

        self.fc_1 = nn.Linear(hid_dim, pf_dim)
        self.fc_2 = nn.Linear(pf_dim, hid_dim)

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        #x = [batch size, seq len, hid dim]

        x = self.dropout(torch.relu(self.fc_1(x)))
        #x = [batch size, seq len, pf dim]

        x = self.fc_2(x)
        #x = [batch size, seq len, hid dim]

        return x
```

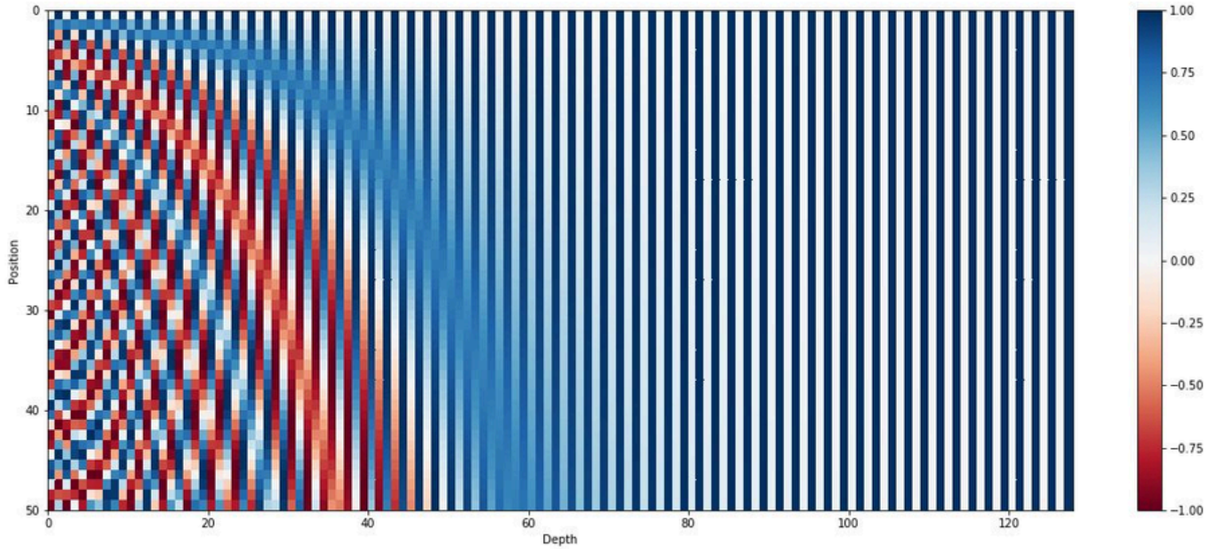


```python
class Encoder(nn.Module):
    def __init__(self,
                 input_dim,
                 hid_dim,
                 n_layers,
                 n_heads,
                 pf_dim,
                 dropout,
                 device,
                 max_length = 100):
        super().__init__()

        self.device = device

        ''' Input Embedding '''
        self.tok_embedding = nn.Embedding(input_dim, hid_dim)
        # <sos> token부터 문장의 최대 길이로 설정한 100까지 positional embedding을 함.
        self.pos_embedding = nn.Embedding(max_length, hid_dim)

        ''' Multiple Encoder '''
        # 논문에서는 N=6개의 encoder layer를 사용하였음
        self.layers = nn.ModuleList([EncoderLayer(hid_dim,
                                                  n_heads,
                                                  pf_dim,
                                                  dropout,
                                                  device)
                                     for _ in range(n_layers)])

        self.dropout = nn.Dropout(dropout)

        self.scale = torch.sqrt(torch.FloatTensor([hid_dim])).to(device)
```



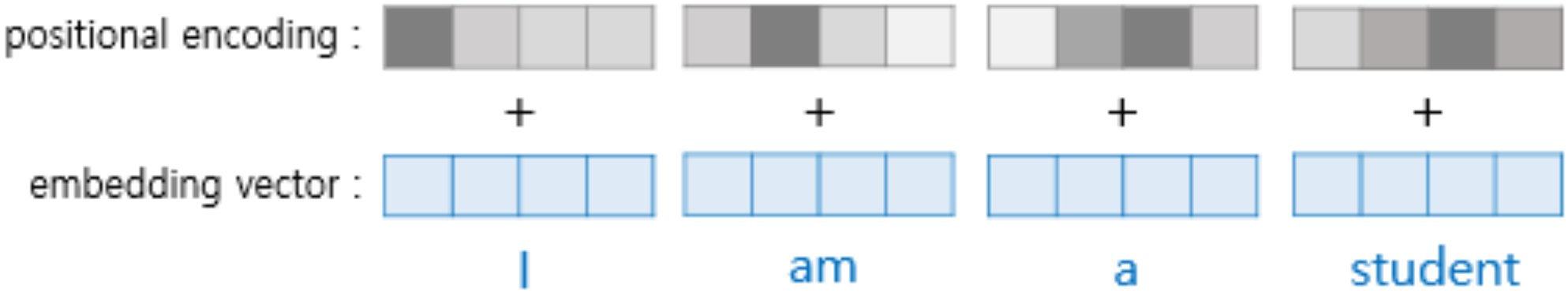

```python
def forward(self, src, src_mask):

    #src = [batch size, src len]
    #src_mask = [batch size, 1, 1, src len]

    batch_size = src.shape[0]
    src_len = src.shape[1]

    pos = torch.arange(0, src_len).unsqueeze(0).repeat(batch_size, 1).to(self.device)

    #pos = [batch size, src len]

    # src 문장의 embedding vector와 positional vector를 더해줌
    src = self.dropout((self.tok_embedding(src) * self.scale) + self.pos_embedding(pos))

    #src = [batch size, src len, hid dim]

    for layer in self.layers:
        src = layer(src, src_mask)

    #src = [batch size, src len, hid dim]

    return src
```

연결관계: positional encoding




```python
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len, device):
        super().__init__()

        # Positional Encoding 초기화
        # 1. 비어있는 tensor 생성
        # (max_len,d_model)
        self.P_E = torch.zeros(max_len, d_model, device=device)
        # 학습되는 값이 아님으로 requires_grad 을 False로 설정
        self.P_E.requires_grad = False

        # 2. pos (0~max_len) 생성 (row 방향 => unsqueeze(dim=1))
        pos = torch.arange(0, max_len, dtype=torch.float, device=device).unsqueeze(dim=1)

        # 3. _2i (0~2i) 생성 (col 방향)
        # 2i는 step = 2 를 활용하여 i의 2배수를 만듦
        _2i = torch.arange(0, d_model, step= 2, dtype=torch.float, device=device)

        # 4. 제안된 positional encoding 생성
        # (i가 짝수일때 : sin, 홀수일때 : cos)
        self.P_E[:, 0::2] = torch.sin(pos / 10000 ** (_2i / d_model))
        self.P_E[:, 1::2] = torch.cos(pos / 10000 ** (_2i / d_model))

    def forward(self,x):
        # x seq 길이에 맞춰 PE return
        # (seq_len, d_model)
        batch_size, seq_len = x.size()
        PE_for_x = self.P_E[:seq_len,:]

        return PE_for_x
```

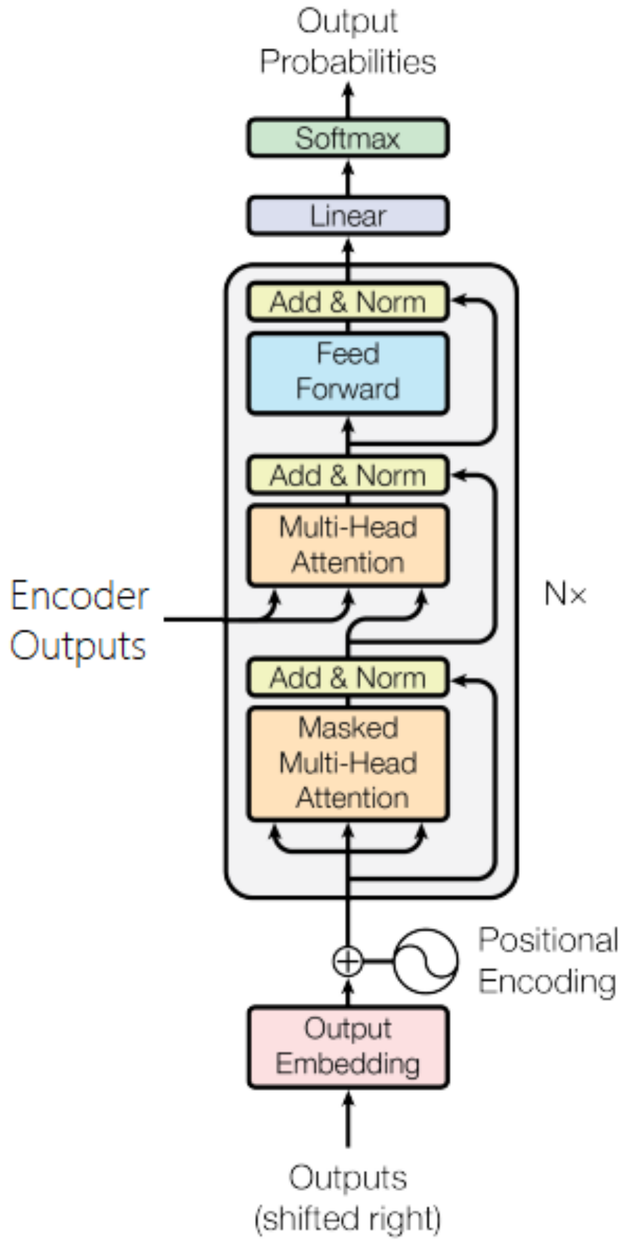
```
# src 문장의 embedding vector와 positional vector를 더해줌
src = self.dropout((self.tok_embedding(src) * self.scale) + self.pos_embedding(pos))
```



더합니다! 붙이는게 아니라!!!

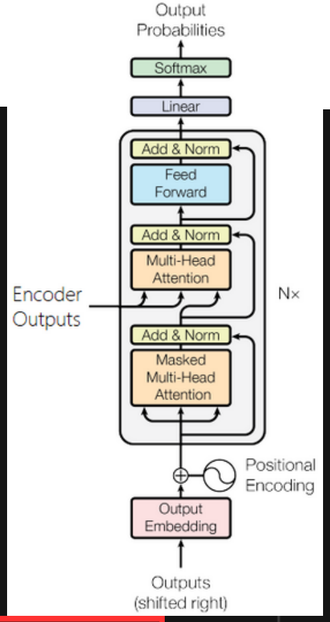
차원의 저주와 함께 생각해보세요!

DECODER

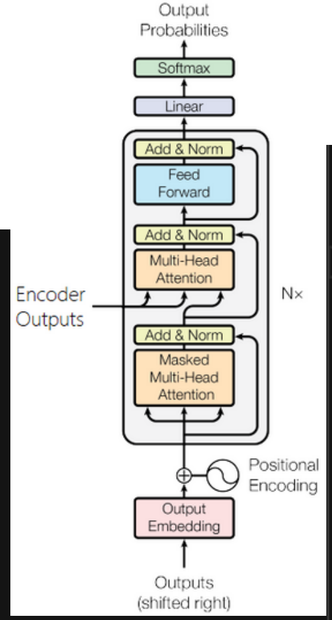

Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Encoder Outputs
Nx
Positional Encoding
Output Embedding
Outputs (shifted right)

```python
class Decoder(nn.Module):
    def __init__(self,
                 output_dim,
                 hid_dim,
                 n_layers,
                 n_heads,
                 pf_dim,
                 dropout,
                 device
                 ):
        super().__init__()

        self.device = device

        self.tok_embedding = nn.Embedding(output_dim, hid_dim)

        self.pos_embedding = nn.Embedding(max_length, hid_dim)

        self.layers = nn.ModuleList([DecoderLayer(hid_dim,
                                                  n_heads,
                                                  pf_dim,
                                                  dropout,
                                                  device)
                                     for _ in range(n_layers)])

        self.fc_out = nn.Linear(hid_dim, output_dim)

        self.dropout = nn.Dropout(dropout)

        self.scale = torch.sqrt(torch.FloatTensor([hid_dim])).to(device)
```

(function) __init__: Any

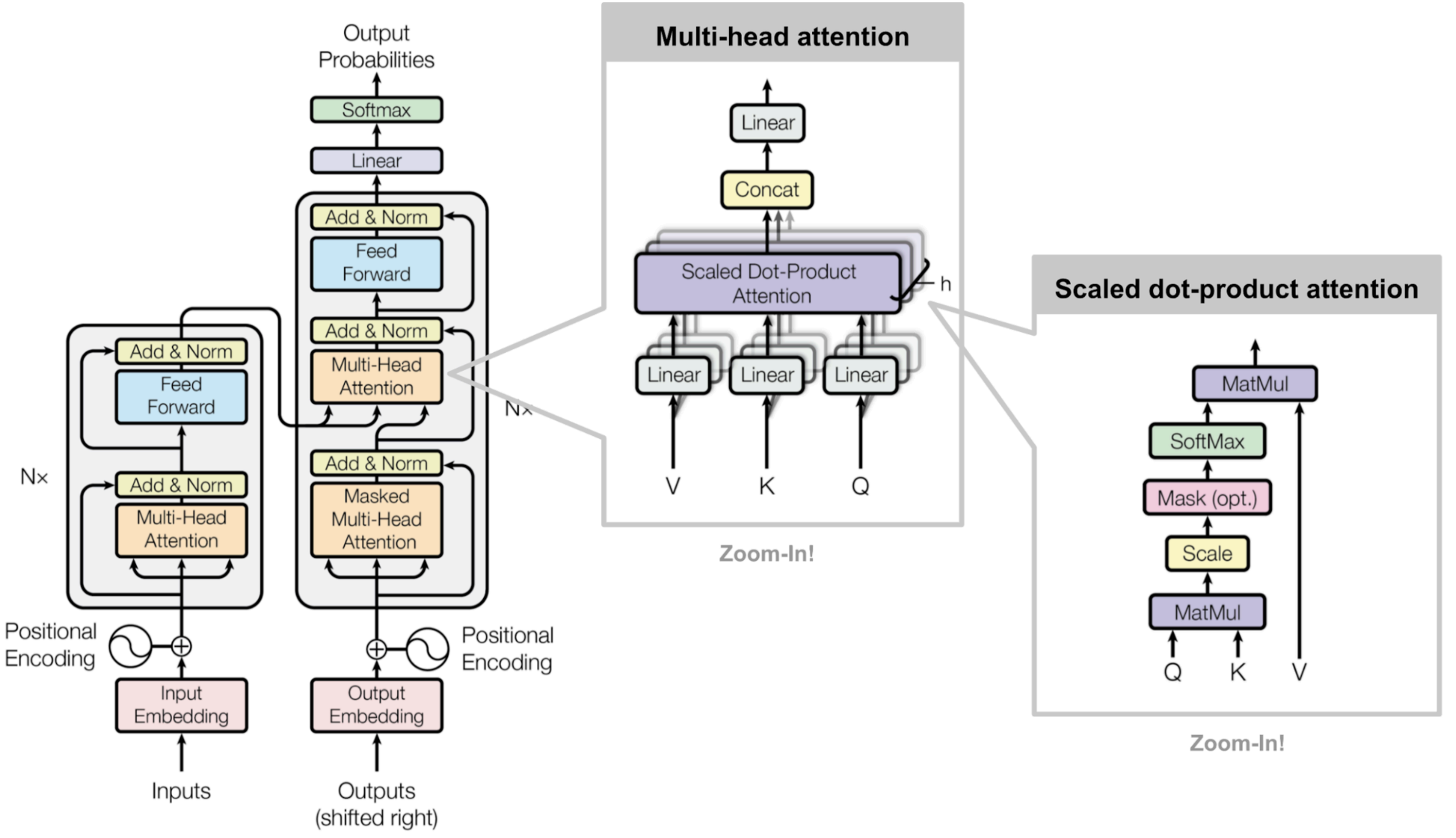



```python
    def forward(self, trg, enc_src, trg_mask, src_mask):

        #trg = [batch size, trg len]
        #enc_src = [batch size, src len, hid dim]
        #trg_mask = [batch size, 1, trg len, trg len]
        #src_mask = [batch size, 1, 1, src len]

        batch_size = trg.shape[0]
        trg_len = trg.shape[1]

        pos = torch.arange(0, trg_len).unsqueeze(0).repeat(batch_size, 1).to(self.device)

        #pos = [batch size, trg len]

        trg = self.dropout((self.tok_embedding(trg) * self.scale) + self.pos_embedding(pos))

        #trg = [batch size, trg len, hid dim]

        for layer in self.layers:
            trg, attention = layer(trg, enc_src, trg_mask, src_mask)

        #trg = [batch size, trg len, hid dim]
        #attention = [batch size, n heads, trg len, src len]

        output = self.fc_out(trg)

        #output = [batch size, trg len, output dim]

        return output, attention
```




```python
class DecoderLayer(nn.Module):
    def __init__(self,
                 hid_dim,
                 n_heads,
                 pf_dim,
                 dropout,
                 device):
        super().__init__()

        ''' Multi Head self Attention'''
        self.self_attention = MultiHeadAttentionLayer(hid_dim, n_heads, dropout, device)
        self.self_attn_layer_norm = nn.LayerNorm(hid_dim)

        ''' Encoder-decoder attention'''
        self.encoder_attention = MultiHeadAttentionLayer(hid_dim, n_heads, dropout, device)
        self.enc_attn_layer_norm = nn.LayerNorm(hid_dim)

        ''' Positionwise FeedForward Layer'''
        self.positionwise_feedforward = PositionwiseFeedforwardLayer(hid_dim,
                                                                      pf_dim,
                                                                      dropout)
        self.ff_layer_norm = nn.LayerNorm(hid_dim)

        self.dropout = nn.Dropout(dropout)
```

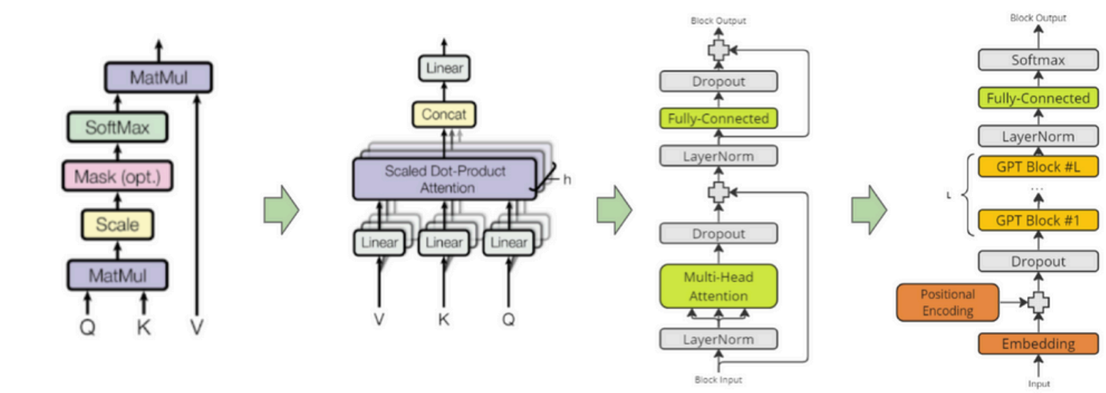
```python
def forward(self, trg, enc_src, trg_mask, src_mask):

    #trg = [batch size, trg len, hid dim]
    #enc_src = [batch size, src len, hid dim]
    #trg_mask = [batch size, 1, trg len, trg len]
    #src_mask = [batch size, 1, 1, src len]

    #self attention
    _trg, _ = self.self_attention(trg, trg, trg, trg_mask)

    #dropout, residual connection and layer norm
    trg = self.self_attn_layer_norm(trg + self.dropout(_trg))

    #trg = [batch size, trg len, hid dim]

    #encoder attention
    _trg, attention = self.encoder_attention(trg, enc_src, enc_src, src_mask)

    #dropout, residual connection and layer norm
    trg = self.enc_attn_layer_norm(trg + self.dropout(_trg))

    #trg = [batch size, trg len, hid dim]

    #positionwise feedforward
    _trg = self.positionwise_feedforward(trg)

    #dropout, residual and layer norm
    trg = self.ff_layer_norm(trg + self.dropout(_trg))

    #trg = [batch size, trg len, hid dim]
    #attention = [batch size, n heads, trg len, src len]

    return trg, attention
```


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
Multi-head attention
Linear
Concat
Scaled Dot-Product Attention
h
Linear
Linear
Linear
V
K
Q
Zoom-In!
Scaled dot-product attention
MatMul
SoftMax
Mask (opt.)
Scale
MatMul
Q
K
V
Zoom-In!

Large Language Model

	Claude 3 Opus	Claude 3 Sonnet	Claude 3 Haiku	GPT-4	GPT-3.5	Gemini 1.0 Ultra	Gemini 1.0 Pro
Undergraduate level knowledge *MMLU*	86.8% 5 shot	79.0% 5-shot	75.2% 5-shot	86.4% 5-shot	70.0% 5-shot	83.7% 5-shot	71.8% 5-shot
Graduate level reasoning *GPQA, Diamond*	50.4% 0-shot CoT	40.4% 0-shot CoT	33.3% 0-shot CoT	35.7% 0-shot CoT	28.1% 0-shot CoT	—	—
Grade school math *GSM8K*	95.0% 0-shot CoT	92.3% 0-shot CoT	88.9% 0-shot CoT	92.0% 5-shot CoT	57.1% 5-shot	94.4% Maj1@32	86.5% Maj1@32
Math problem-solving *MATH*	60.1% 0-shot CoT	43.1% 0-shot CoT	38.9% 0-shot CoT	52.9% 4-shot	34.1% 4-shot	53.2% 4-shot	32.6% 4-shot
Multilingual math *MGSM*	90.7% 0-shot	83.5% 0-shot	75.1% 0-shot	74.5% 8-shot	—	79.0% 8-shot	63.5% 8-shot
Code *HumanEval*	84.9% 0-shot	73.0% 0-shot	75.9% 0-shot	67.0% 0-shot	48.1% 0-shot	74.4% 0-shot	67.7% 0-shot
Reasoning over text *DROP, F1 score*	83.1 3-shot	78.9 3-shot	78.4 3-shot	80.9 3-shot	64.1 3-shot	82.4 Variable shots	74.1 Variable shots
Mixed evaluations *BIG-Bench-Hard*	86.8% 3-shot CoT	82.9% 3-shot CoT	73.7% 3-shot CoT	83.1% 3-shot CoT	66.6% 3-shot CoT	83.6% 3-shot CoT	75.0% 3-shot CoT
Knowledge Q&A *ARC-Challenge*	96.4% 25-shot	93.2% 25-shot	89.2% 25-shot	96.3% 25-shot	85.2% 25-shot	—	—
Common Knowledge *HellaSwag*	95.4% 10-shot	89.0% 10-shot	85.9% 10-shot	95.3% 10-shot	85.5% 10-shot	87.8% 10-shot	84.7% 10-shot

텍스트 처리 결과



Scale Causal Attention | Multi-Head Attention | Transformer Block | GPT Model

	Claude 3 Opus	Claude 3 Sonnet	Claude 3 Haiku	GPT-4V	Gemini 1.0 Ultra	Gemini 1.0 Pro
Math & reasoning *MMMU (val)*	59.4%	53.1%	50.2%	56.8%	59.4%	47.9%
Document visual Q&A *ANLS score, test*	89.3%	89.5%	88.8%	88.4%	90.9%	88.1%
Math *MathVista (testmini)*	50.5% CoT	47.9% CoT	46.4% CoT	49.9%	53.0%	45.2%
Science diagrams *AI2D, test*	88.1%	88.7%	86.7%	78.2%	79.5%	73.9%
Chart Q&A *Relaxed accuracy (test)*	80.8% 0-shot CoT	81.1% 0-shot CoT	81.7% 0-shot CoT	78.5% 4-shot CoT	80.8%	74.1%

시각 처리 결과

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE (2021)

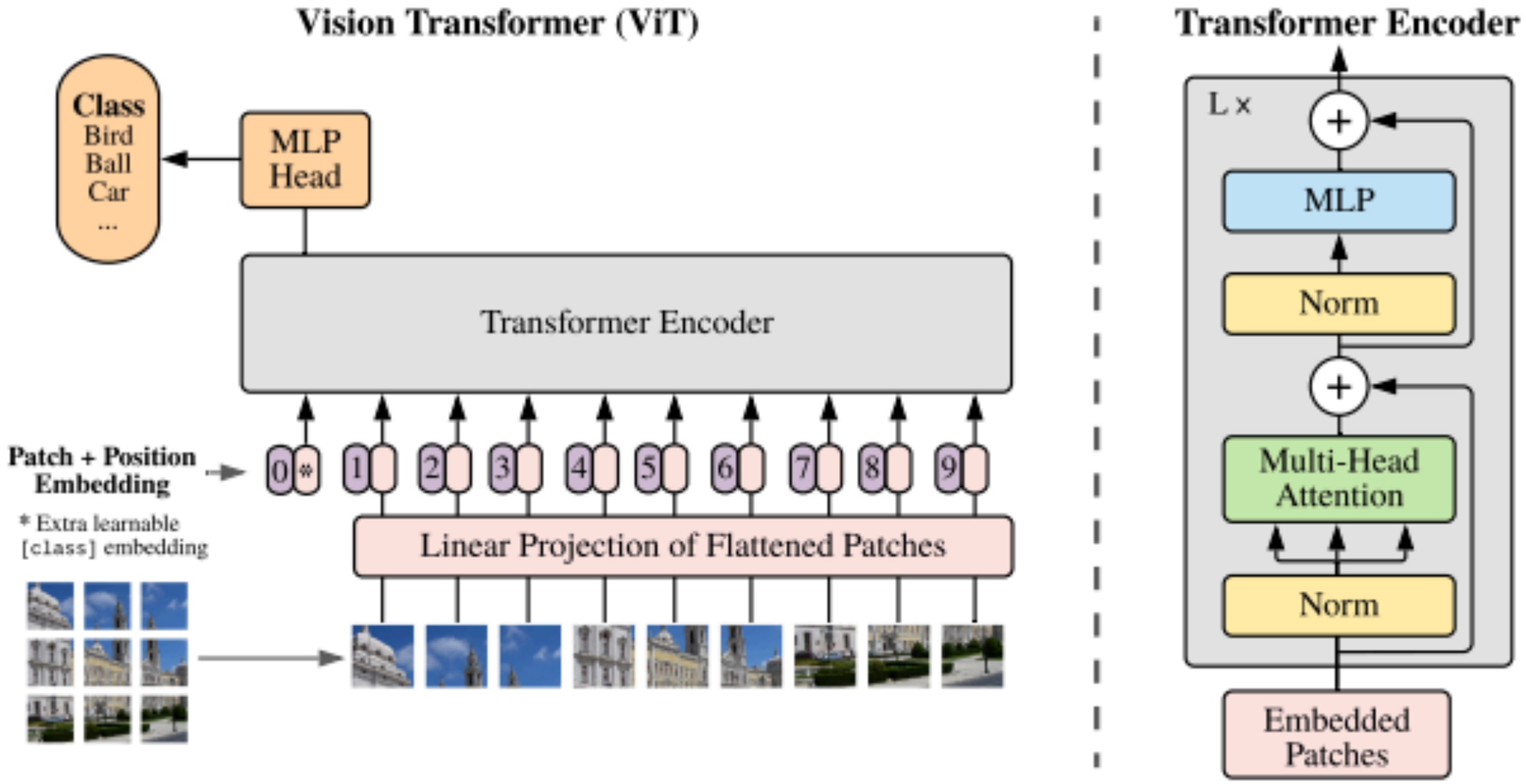


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable "classification token" to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

DEEP DOUBLE DESCENT:WHERE BIGGER MODELS AND MORE DATA HURT

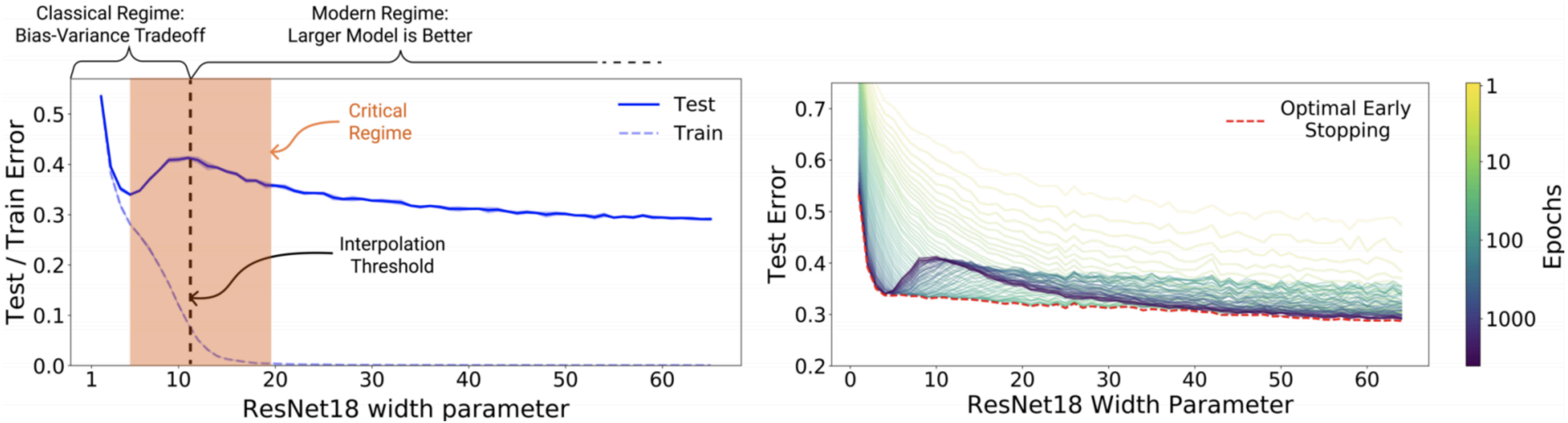
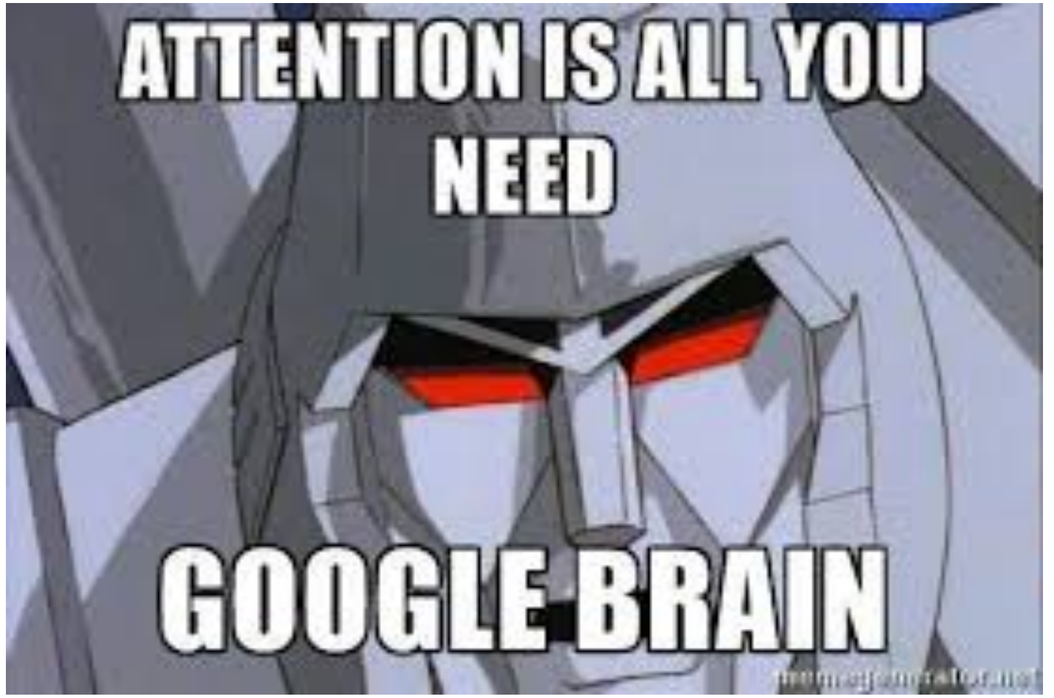


Figure 1: **Left:** Train and test error as a function of model size, for ResNet18s of varying width on CIFAR-10 with 15% label noise. **Right:** Test error, shown for varying train epochs. All models trained using Adam for 4K epochs. The largest model (width 64) corresponds to standard ResNet18.

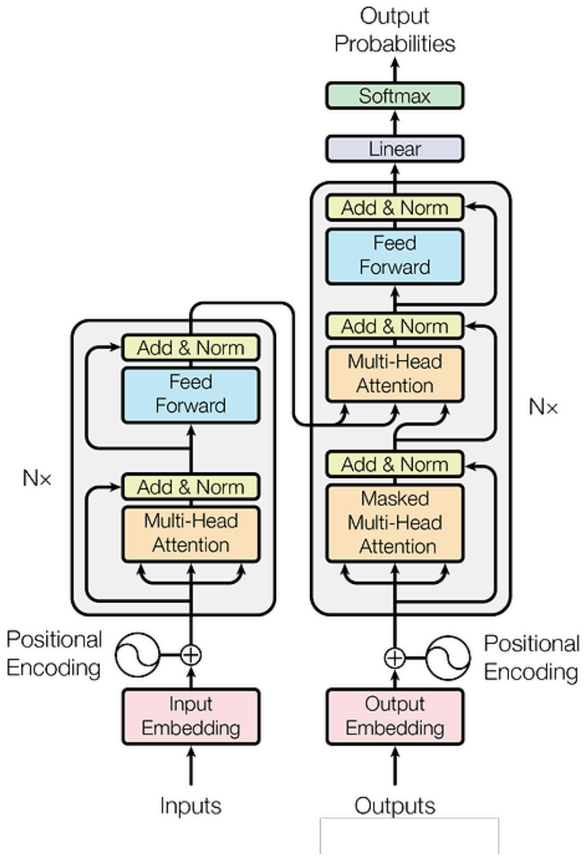
LARGER MODEL IS BETTER...


ATTENTION IS ALL YOU NEED
GOOGLE BRAIN

So, how do we improve Neural Machine Translation?
Use ConvS2S
ByteNet variant?
ONLY Attention

BERT

Encoder



GPT

Decoder

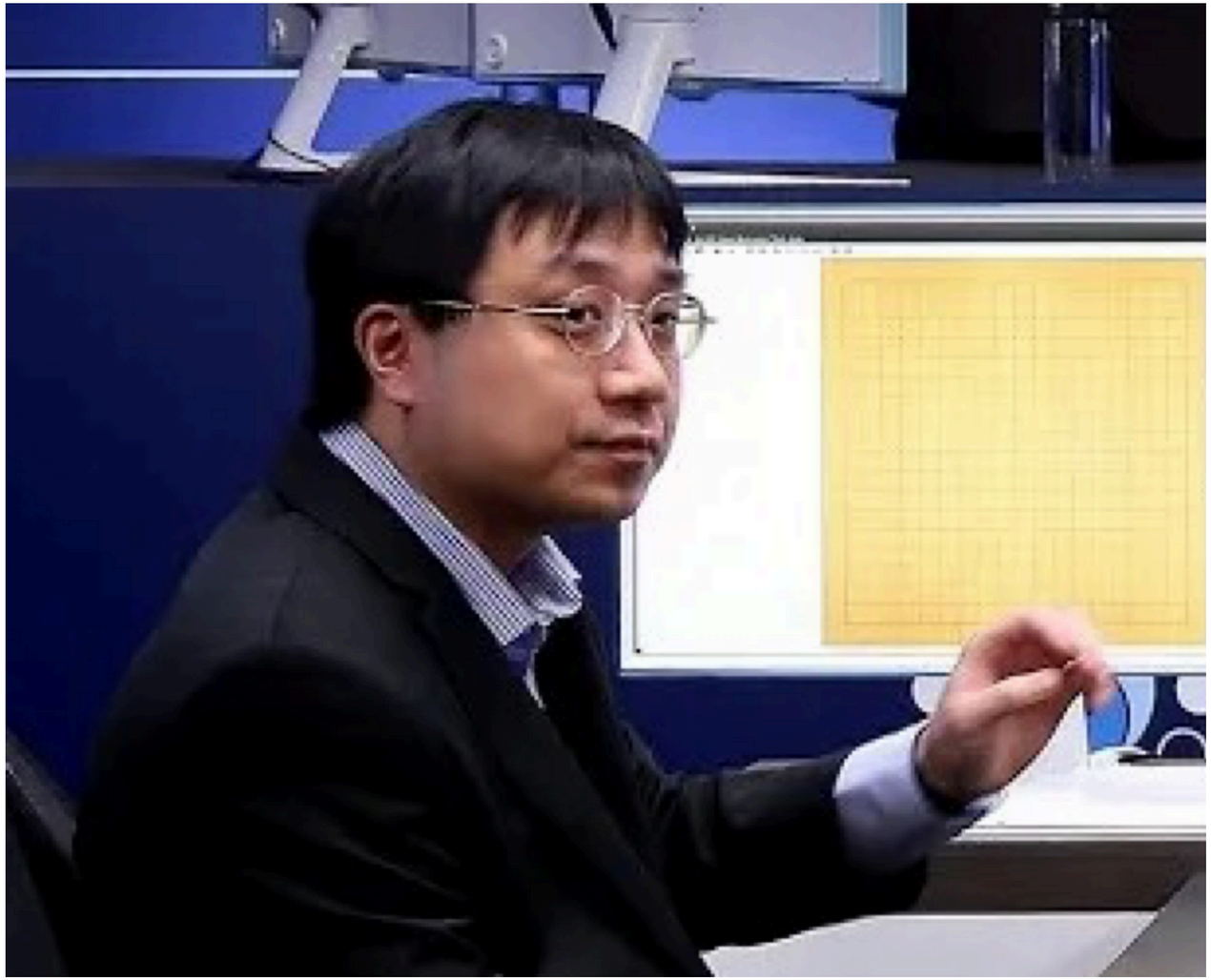
GPT

어제 카페 갔었어 거기 사람 많더라

BERT

어제 카페 갔었어 사람 많더라

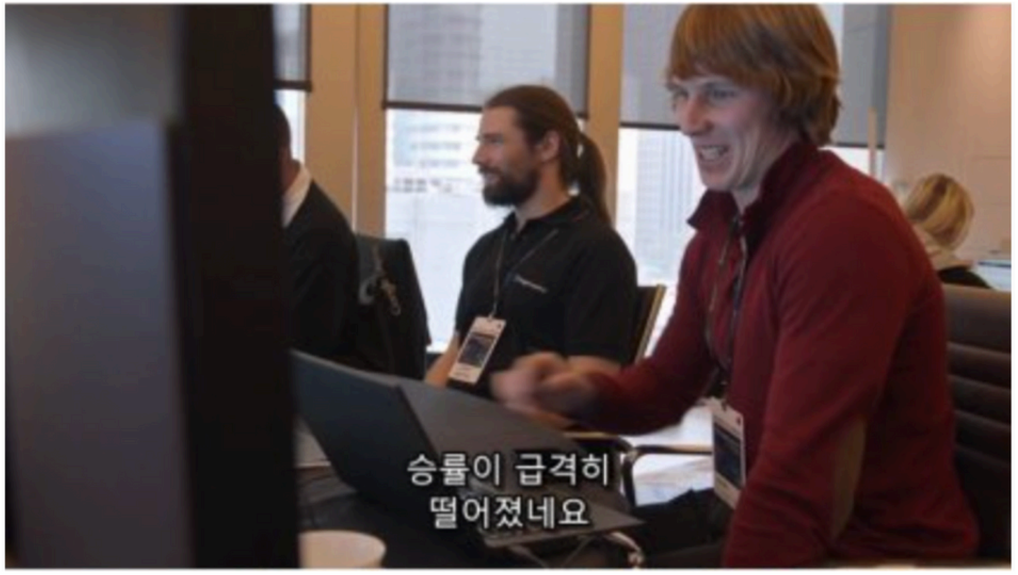
강화학습

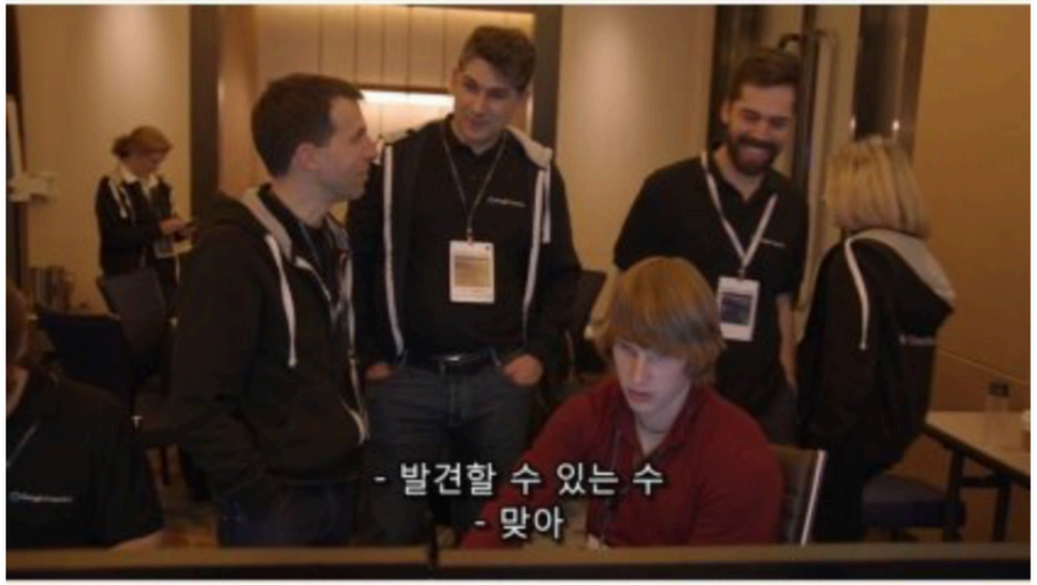


ALPHAGO
02:00:00
LEE SEDOL
01:59:41
AlphaGo
Lee Sedol
sina 新浪体育

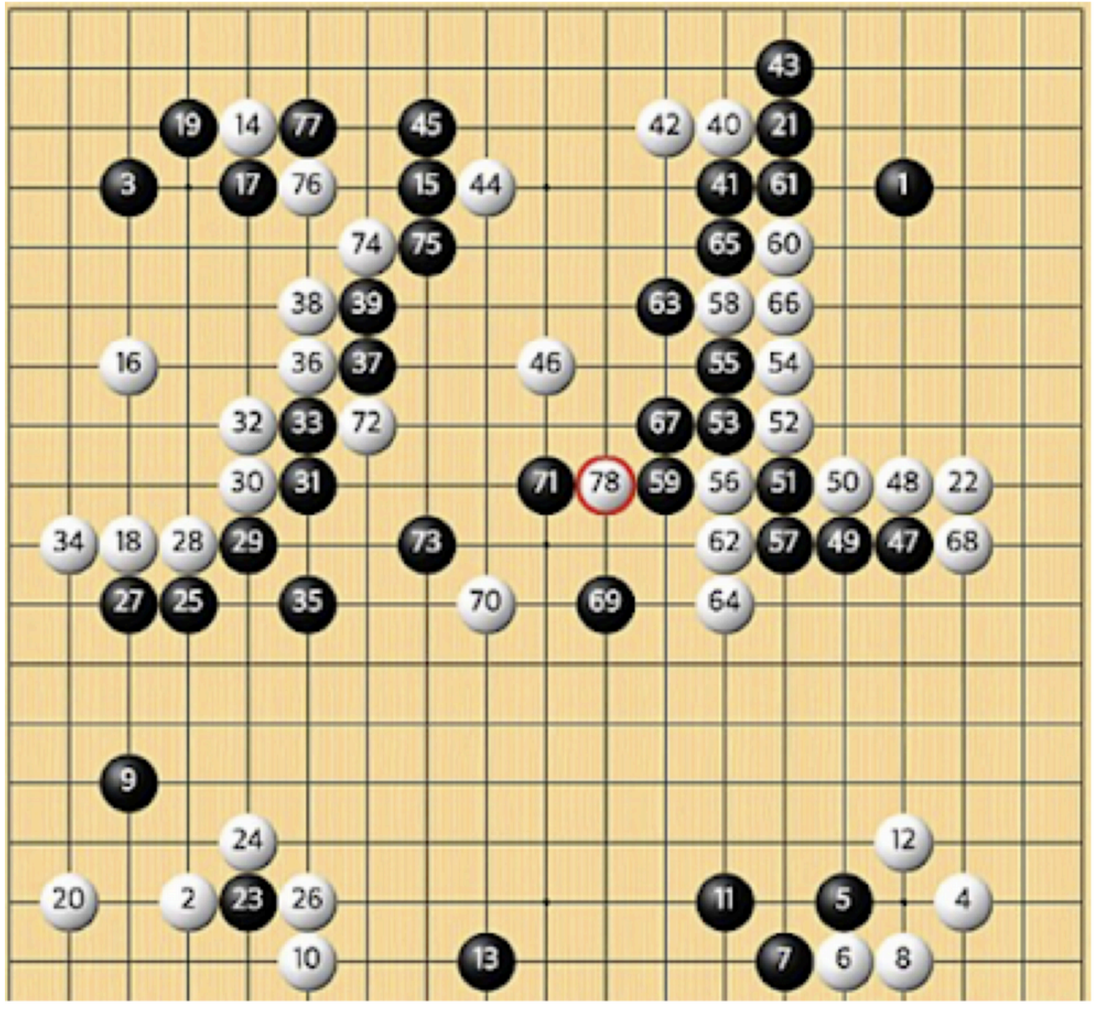
초 집중 상태입니다

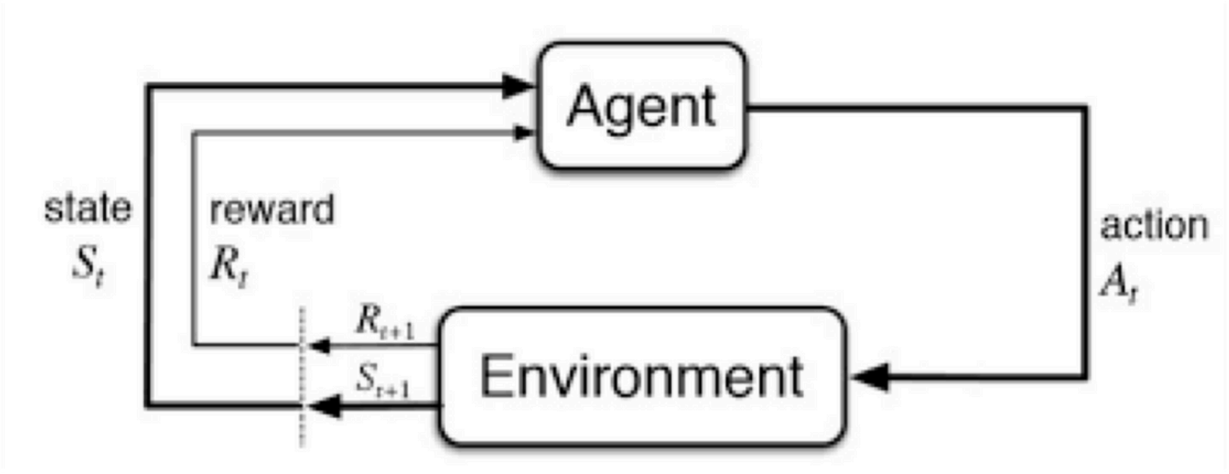
1만명 중에
오직 한 명만이

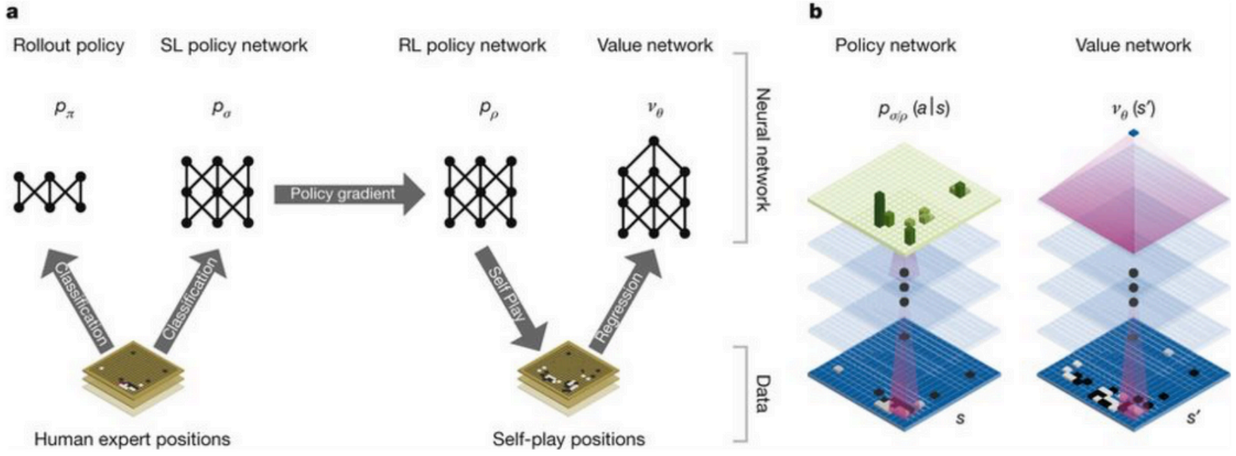

승률이 급격히
떨어졌네요


- 발견할 수 있는 수
- 맞아

강화학습?!


43
19 14 77 45 42 40 21
3 17 76 15 44 41 61 1
74 75 65 60
38 39 63 58 66
16 36 37 46 55 54
32 33 72 67 53 52
30 31 71 78 59 56 51 50 48 22
34 18 28 29 73 62 57 49 47 68
27 25 35 70 69 64
9
24 12
20 2 23 26 11 5 4
10 13 7 6 8


Agent
state
S_t
reward
R_t
action
A_t
R_{t+1}
S_{t+1}
Environment

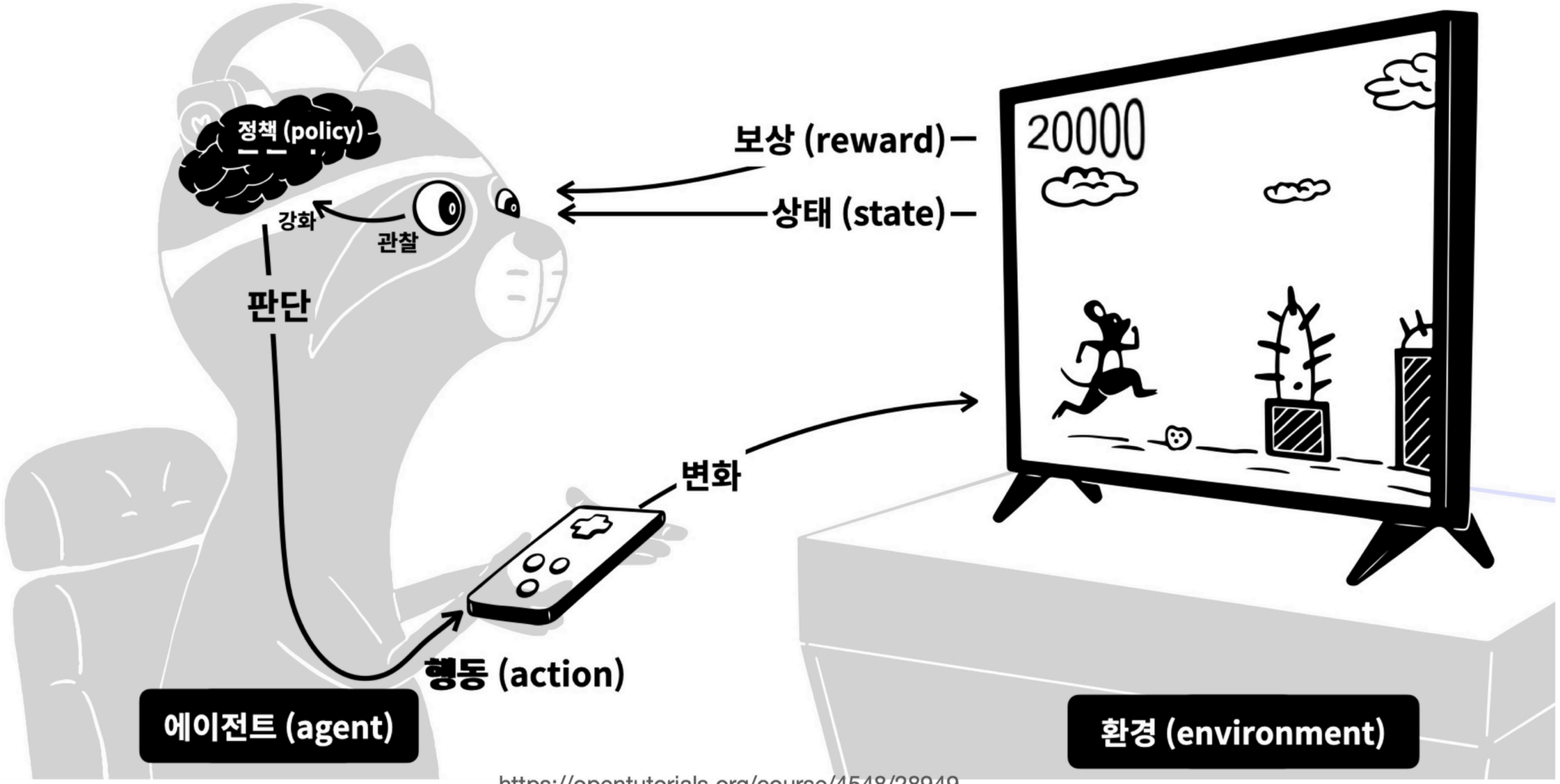

a
Rollout policy
SL policy network
RL policy network
Value network
p_π
p_σ
p_ρ
v_θ
Policy gradient
Classification
Classification
Self Play
Regression
Neural network
Data
Human expert positions
Self-play positions
b
Policy network
Value network
$p_{\sigma/\rho}(a|s)$
$v_\theta(s')$
s
s'

어떤느낌?!

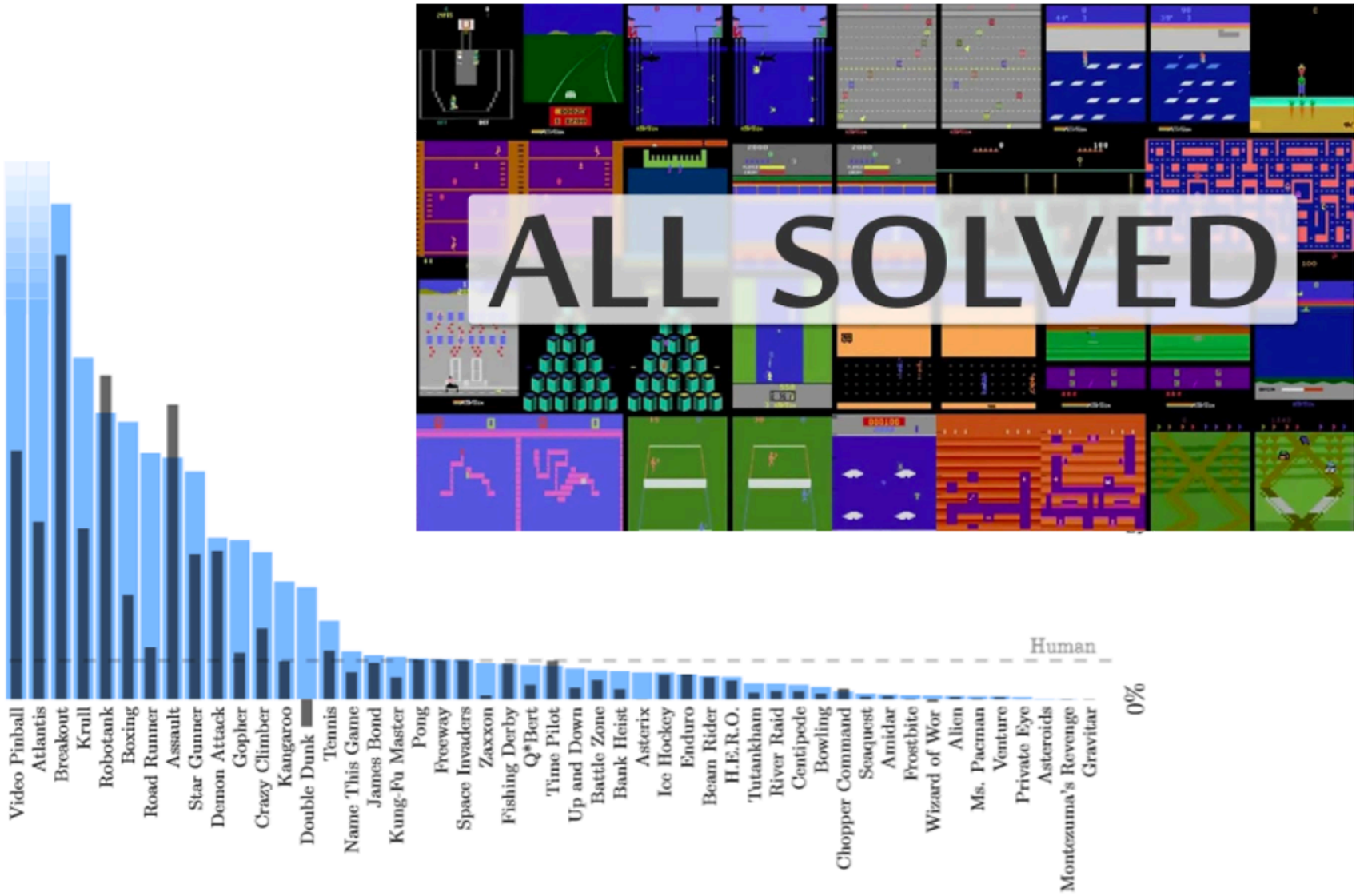
움…

그러니까…

이렇게!!



https://opentutorials.org/course/4548/28949



사람보다 게임을 잘 하는 Atari

Credit: Nvidia developer, youtube 'two minutes paper'